Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №6
дисциплины
«Основы нейронных сетей»
Вариант № 14

Выполнила:
Текеева Мадина Азрет-Алиевна
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

_____
(подпись)

Проверил:
Доцент департамента цифровых, робототехнических систем и электроники института перспективной инженерии Воронкин Р.А

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты_____

Ставрополь, 2026 г.

Тема: Изучение методов регрессии и оценки качества моделей нейронных сетей для предсказания числовых значений.

Цель работы: освоение методов построения, обучения и сравнения полносвязных нейронных сетей для задачи регрессии на реальных данных с использованием библиотек TensorFlow/Keras и PyTorch.

Репозиторий: https://github.com/bickrosss/NN_lab6

Порядок выполнения работы:

Задание 1.1. Реализация средствами библиотеки TensorFlow/Keras.

1. Импорты.

Подключены все необходимые библиотеки: NumPy и Pandas для работы с данными, модули Keras для построения нейросетей, оптимизаторы и коллбэки, scikit-learn для нормализации и метрики MAE, а также gdown для загрузки датасета и matplotlib для визуализации. Установлен глобальный seed (42) для воспроизводимости результатов.

```
# Работа с массивами и таблицами
import numpy as np
import pandas as pd

# Keras
from tensorflow.keras import utils
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import (Dense, Dropout, BatchNormalization,
                                     Input, concatenate, Activation)
from tensorflow.keras.optimizers import Adam, SGD, RMSprop, Adagrad
from tensorflow.keras.callbacks import LambdaCallback

# Прочее
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error
import gdown
import re
import matplotlib.pyplot as plt

%matplotlib inline
np.random.seed(42)
import tensorflow as tf
tf.random.set_seed(42)
```

Рисунок 1.1 – Импорт библиотек (Keras)

2. Подготовка данных.

Датасет загружен из облачного хранилища (CSV-файл с HeadHunter). Выполнена очистка строк от спецсимволов, парсинг числовых столбцов и сбор

итогового числового вектора признаков длиной 39 элементов. Исходный датафрейм содержит 62 967 строк и 12 столбцов.

```python
# Скачиваем базу
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l10/hh_fixed.csv',
               None, quiet=True)

# Чтение
df = pd.read_csv('hh_fixed.csv', index_col=0)
print('Размер исходного датафрейма:', df.shape)
df.head(3)
```

```
Размер исходного датафрейма: (62967, 12)
```

	Пол, возраст	ЗП	Ищет работу на должность:	Город	Занятость	График	Опыт (двойное нажатие для полной версии)	Последенее/ нынешнее место работы	Последеняя/ нынешняя должность	Образование и ВУЗ	Обновление резюме	Авто
0	Мужчина , 29 лет , родился 16 мая 1989	40000 руб.	Специалист по поддержке чата(support team) дом...	Новороссийск , готов к переезду (Анапа, Геленд...	полная занятость	полный день	Опыт работы 3 года 9 месяцев Специалист по по...	ООО "Гольфстрим"	Генеральный директор	Высшее образование 2011 Международный юридиче...	26.04.2019 08:04	Не указано
1	Мужчина , 38 лет , родился 25 мая 1980	40000 руб.	Системный администратор	Новосибирск , м. Березовая роща , не готов к ...	полная занятость	полный день	Опыт работы 11 лет 11 месяцев Системный админ...	ООО «Завод модульных технологий»	Системный администратор	Высшее образование 2002 Новосибирский государс...	26.04.2019 04:30	Не указано
2	Мужчина , 35 лет , родился 14 июня 1983	300000 руб.	DevOps TeamLead / DevOps архитектор	Москва , готов к переезду , готов к редким ком...	полная занятость	полный день	Опыт работы 12 лет 11 месяцев DevOps TeamLead...	Банк ВТБ (ПАО)	Начальник отдела методологии разработки (DevOp...	DevOps TeamLead / DevOps архитектор 300 000 ру...	09.04.2019 14:40	Не указано

```python
# Номера столбцов
COL_SEX_AGE  = df.columns.get_loc('Пол, возраст')
COL_SALARY   = df.columns.get_loc('ЗП')
COL_POS_SEEK = df.columns.get_loc('Ищет работу на должность:')
COL_POS_PREV = df.columns.get_loc('Последеняя/нынешняя должность')
COL_CITY     = df.columns.get_loc('Город')
COL_EMPL     = df.columns.get_loc('Занятость')
COL_SCHED    = df.columns.get_loc('График')
COL_EXP      = df.columns.get_loc('Опыт (двойное нажатие для полной версии)')
COL_EDU      = df.columns.get_loc('Образование и ВУЗ')
COL_UPDATED  = df.columns.get_loc('Обновление резюме')
```

Рисунок 1.2 – Подготовка датасета

```python
# Очистка строк
def purify(x):
    if isinstance(x, str):
        x = x.replace('\n', ' ').replace('\xa0', '').strip().lower()
    return x

# Применяем
for col in df.columns:
    df[col] = df[col].apply(purify)

# Извлечение года из даты
def extract_year(x):
    try:
        return int(re.search(r'\d\d.\d\d.(\d{4})', x)[1])
    except (IndexError, TypeError, ValueError):
        return 0
```

```python
# Параметрические данные
currency_rate = {'usd': 65., 'kzt': 0.17, 'грн': 2.6, 'белруб': 30.5,
                 'eur': 70., 'kgs': 0.9, 'сум': 0.007, 'azn': 37.5}

age_class        = [0, [18, 23, 28, 33, 38, 43, 48, 53, 58, 63]]
experience_class = [0, [7, 13, 25, 37, 61, 97, 121, 157, 193, 241]]

city_class = [0,
              {'москва': 0, 'санкт-петербург': 1,
               'новосибирск': 2, 'екатеринбург': 2, 'нижний новгород': 2,
               'казань': 2, 'челябинск': 2, 'омск': 2, 'самара': 2,
               'ростов-на-дону': 2, 'уфа': 2, 'красноярск': 2, 'пермь': 2,
               'воронеж': 2, 'волгоград': 2,
               'прочие города': 3}]

employment_class = [0, {'стажировка': 0, 'частичная занятость': 1,
                        'проектная работа': 2, 'полная занятость': 3}]

schedule_class   = [0, {'гибкий график': 0, 'полный день': 1,
                        'сменный график': 2, 'удаленная работа': 3}]

education_class  = [0, {'высшее образование': 0, 'higher education': 0,
                        'среднее специальное': 1, 'неоконченное высшее': 2,
                        'среднее образование': 3}]

# Подсчёт количества классов
for class_lst in [age_class, experience_class]:
    class_lst[0] = len(class_lst[1]) + 1
for class_dct in [city_class, employment_class, schedule_class, education_class]:
    class_dct[0] = max(class_dct[1].values()) + 1

print('Возраст -', age_class[0], 'классов')
print('Опыт -', experience_class[0], 'классов')
print('Города -', city_class[0], 'классов')
```

```
Возраст - 11 классов
Опыт - 11 классов
Города - 4 классов
```

Рисунок 1.3 – Очистка строк, параметрические данные и подсчёт классов

```python
# One-hot для целочисленного значения
def int_to_ohe(arg, class_list):
    num_classes = class_list[0]
    for i in range(num_classes - 1):
        if arg < class_list[1][i]:
            cls = i
            break
    else:
        cls = num_classes - 1
    return utils.to_categorical(cls, num_classes)

# Multi-hot из строки и словаря
def str_to_multi(arg, class_dict):
    num_classes = class_dict[0]
    result = np.zeros(num_classes)
    for value, cls in class_dict[1].items():
        if value in arg:
            result[cls] = 1.
    return result
```

```python
# Парсинг пола и возраста
base_update_year = 2019

def extract_sex_age_years(arg):
    sex = 1. if 'муж' in arg else 0.
    try:
        years = base_update_year - int(re.search(r'\d{4}', arg)[0])
    except (IndexError, TypeError, ValueError):
        years = 0
    return sex, years

def age_years_to_ohe(arg):
    return int_to_ohe(arg, age_class)

def experience_months_to_ohe(arg):
    return int_to_ohe(arg, experience_class)

# Парсинг зарплаты
def extract_salary(arg):
    try:
        value = float(re.search(r'\d+', arg)[0])
        for currency, rate in currency_rate.items():
            if currency in arg:
                value *= rate
                break
    except TypeError:
        value = 0.
    return value / 1000.   # тыс. рублей
```

Рисунок 1.4 – Функции кодирования: int_to_ohe, str_to_multi, парсинг пола/возраста/зарплаты

```python
# Город -> one-hot
def extract_city_to_ohe(arg):
    num_classes = city_class[0]
    split_array = re.split(r'[ ,.:()?!]', arg)
    for word in split_array:
        city_cls = city_class[1].get(word, -1)
        if city_cls >= 0:
            break
    else:
        city_cls = num_classes - 1
    return utils.to_categorical(city_cls, num_classes)

# Multi-hot для занятости / графика / образования
def extract_employment_to_multi(arg):
    return str_to_multi(arg, employment_class)

def extract_schedule_to_multi(arg):
    return str_to_multi(arg, schedule_class)

def extract_education_to_multi(arg):
    result = str_to_multi(arg, education_class)
    if result[2] > 0.:          # неоконченное высшее => убираем «высшее»
        result[0] = 0.
    return result

# Опыт работы (месяцев)
def extract_experience_months(arg):
    try:
        years = int(re.search(r'(\d+)\s+(год.?|лет)', arg)[1])
    except (IndexError, TypeError, ValueError):
        years = 0
    try:
        months = int(re.search(r'(\d+)\s+месяц', arg)[1])
    except (IndexError, TypeError, ValueError):
        months = 0
    return years * 12 + months
```

```python
# Сборка одной строки
def extract_row_data(row):
    sex, age = extract_sex_age_years(row[COL_SEX_AGE])
    sex_vec   = np.array([sex])
    age_ohe   = age_years_to_ohe(age)
    city_ohe  = extract_city_to_ohe(row[COL_CITY])
    empl_mult = extract_employment_to_multi(row[COL_EMPL])
    sched_mlt = extract_schedule_to_multi(row[COL_SCHED])
    edu_multi = extract_education_to_multi(row[COL_EDU])
    exp_mon   = extract_experience_months(row[COL_EXP])
    exp_ohe   = experience_months_to_ohe(exp_mon)
    salary    = extract_salary(row[COL_SALARY])
    salary_v  = np.array([salary])

    x_data = np.hstack([sex_vec, age_ohe, city_ohe,
                        empl_mult, sched_mlt, edu_multi, exp_ohe])
    return x_data, salary_v

# Сбор всей выборки
def construct_train_data(row_list):
    x_data, y_data = [], []
    for row in row_list:
        x, y = extract_row_data(row)
        if y[0] > 0:
            x_data.append(x)
            y_data.append(y)
    return np.array(x_data), np.array(y_data)
```

Рисунок 1.5 – Функции извлечения города, занятости, графика, образования, опыта; сборка строки и выборки

```python
# Формируем выборку
x_train_01, y_train = construct_train_data(df.values)
print('x_train_01.shape =', x_train_01.shape)
print('y_train.shape    =', y_train.shape)
print('Пример x:', x_train_01[0])
print('Пример y:', y_train[0])
```

```
x_train_01.shape = (62967, 39)
y_train.shape    = (62967, 1)
Пример x: [1. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 1. 0. 1. 0. 0.
 1. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]
Пример y: [40.]
```

Рисунок 1.6 – Формирование обучающей выборки

3. Нормализация целевой переменной и валидационная выборка.

Целевая переменная (зарплата) нормирована с помощью StandardScaler. После каждого обучения предсказание денормируется, ошибка считается в тысячах рублей. Для валидации случайно выбрано ~15% строк. Итого: Train – 53 543, Val – 9 424.

```
# Нормализация y
y_scaler = StandardScaler()
y_train_scaled = y_scaler.fit_transform(y_train)

print('mean =', round(y_train_scaled.mean(), 4),
      ', std =', round(y_train_scaled.std(), 4))
```

```
mean = -0.0 , std = 1.0
```

```
# Маска для валидации (~15%)
np.random.seed(42)
val_mask = np.random.rand(len(x_train_01)) < 0.15
print('Train:', (~val_mask).sum(), '   Val:', val_mask.sum())

x_tr,  x_val  = x_train_01[~val_mask], x_train_01[val_mask]
y_tr,  y_val  = y_train_scaled[~val_mask], y_train_scaled[val_mask]
y_val_orig    = y_train[val_mask]  # для финальной денормированной MAE
```

```
Train: 53543    Val: 9424
```

Рисунок 1.7 – Нормализация y и формирование валидационной маски

4. Вспомогательные функции.

Реализованы: plot_history – кривые MAE по эпохам; eval_net_scaled – итоговая MAE в тыс. руб.; make_rub_callback – LambdaCallback для вывода MAE в реальных рублях после каждой эпохи.

```
def plot_history(history, title=''):
    plt.figure(figsize=(8, 4))
    plt.plot(history.history['mae'],     label='MAE train')
    plt.plot(history.history['val_mae'], label='MAE val')
    plt.title(title)
    plt.xlabel('Эпоха обучения')
    plt.ylabel('MAE (нормированная)')
    plt.legend(); plt.grid(); plt.show()

def eval_net_scaled(model, x_val, y_val_orig):
    pred_scaled = model.predict(x_val, verbose=0)
    pred        = y_scaler.inverse_transform(pred_scaled)
    mae         = mean_absolute_error(y_val_orig, pred)
    return mae, pred
```

```
# Callback по теории — выводит MAE в тыс. руб. в конце каждой эпохи
def make_rub_callback(model, x_val, y_val_orig, store):
    def on_epoch_end(epoch, logs):
        pred = y_scaler.inverse_transform(model.predict(x_val, verbose=0))
        mae  = float(np.mean(np.abs(pred - y_val_orig)))
        store.append(mae)
        if (epoch + 1) % 10 == 0 or epoch == 0:
            print(f'  Эпоха {epoch+1:3d} | MAE val (тыс. руб.) = {mae:6.2f}')
    return LambdaCallback(on_epoch_end=on_epoch_end)
```

```
# Здесь будут результаты всех 6 моделей
results = []  # [{name, params, mae, history, rub_history}]
```

Рисунок 1.8 – Вспомогательные функции Keras

5. Обучение и сравнение шести моделей (Keras).

Модель 1 – базовая (минимальная).

Гиперпараметры: 2 слоя по 64/32, активация relu, Adam(lr=1e-3), batch_size=128, 30 эпох.

```
name1 = 'M1 baseline (Dense 64-32, Adam 1e-3)'
print('=== ' + name1 + ' ===')

m1 = Sequential([
    Dense(64, activation='relu', input_dim=x_train_01.shape[1]),
    Dense(32, activation='relu'),
    Dense(1,  activation='linear'),
])
m1.compile(optimizer=Adam(learning_rate=1e-3), loss='mse', metrics=['mae'])

rub1 = []
h1 = m1.fit(x_tr, y_tr,
            validation_data=(x_val, y_val),
            epochs=30, batch_size=128, verbose=0,
            callbacks=[make_rub_callback(m1, x_val, y_val_orig, rub1)])

mae1, _ = eval_net_scaled(m1, x_val, y_val_orig)
print(f'\nИтоговый MAE (тыс. руб.): {mae1:.2f}')
plot_history(h1, name1)

results.append({'name': name1,
                'params': '2×Dense (64,32), Adam 1e-3, bs=128, ep=30',
                'mae': mae1, 'history': h1, 'rub_history': rub1})
```

```
=== M1 baseline (Dense 64-32, Adam 1e-3) ===
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`i
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
  Эпоха   1 | MAE val (тыс. руб.) =  33.15
  Эпоха  10 | MAE val (тыс. руб.) =  31.80
  Эпоха  20 | MAE val (тыс. руб.) =  31.53
  Эпоха  30 | MAE val (тыс. руб.) =  31.62

Итоговый MAE (тыс. руб.): 31.62
```

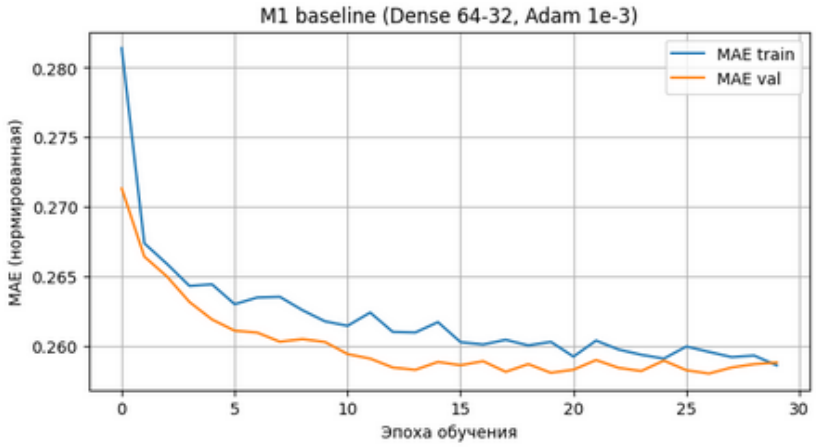


Рисунок 1.9 – Модель 1 Keras: обучение и кривые MAE. Итоговый MAE: 31.62 тыс. руб.

Модель 2 – глубокая с BatchNormalization.

Гиперпараметры: 4 скрытых слоя (256→128→64→32), BN после входа, Adam(lr=1e-3), batch_size=256, 50 эпох.

```python
name2 = 'M2 deep+BN (256-128-64-32, Adam 1e-3)'
print('=== ' + name2 + ' ===')

m2 = Sequential([
    BatchNormalization(input_dim=x_train_01.shape[1]),
    Dense(256, activation='relu'),
    Dense(128, activation='relu'),
    Dense(64,  activation='relu'),
    Dense(32,  activation='relu'),
    Dense(1,   activation='linear'),
])
m2.compile(optimizer=Adam(learning_rate=1e-3), loss='mse', metrics=['mae'])

rub2 = []
h2 = m2.fit(x_tr, y_tr,
            validation_data=(x_val, y_val),
            epochs=50, batch_size=256, verbose=0,
            callbacks=[make_rub_callback(m2, x_val, y_val_orig, rub2)])

mae2, _ = eval_net_scaled(m2, x_val, y_val_orig)
print(f'\nИтоговый MAE (тыс. руб.): {mae2:.2f}')
plot_history(h2, name2)

results.append({'name': name2,
                'params': 'BN + 4 Dense (256,128,64,32), Adam 1e-3, bs=256, ep=50',
                'mae': mae2, 'history': h2, 'rub_history': rub2})
```

```
=== M2 deep+BN (256-128-64-32, Adam 1e-3) ===
/usr/local/lib/python3.12/dist-packages/keras/src/layers/normalization/batch_normalization.py:142: UserWarning: Do not pass an
  super().__init__(**kwargs)
  Эпоха   1 | MAE val (тыс. руб.) =  33.73
  Эпоха  10 | MAE val (тыс. руб.) =  31.21
  Эпоха  20 | MAE val (тыс. руб.) =  31.72
  Эпоха  30 | MAE val (тыс. руб.) =  32.29
  Эпоха  40 | MAE val (тыс. руб.) =  32.44
  Эпоха  50 | MAE val (тыс. руб.) =  32.49

Итоговый MAE (тыс. руб.): 32.49
```

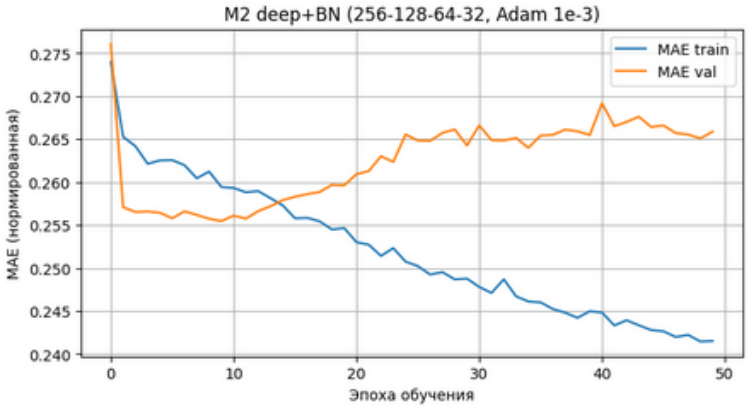


Рисунок 1.10 – Модель 2 Keras: обучение и кривые MAE. Итоговый MAE: 32.49 тыс. руб.

Модель 3 – широкая (1 большой скрытый слой).

Гиперпараметры: Dense(1024, relu) → Dense(64), Adam(lr=5e-4), batch_size=512, 40 эпох.

```python
name3 = 'M3 wide (1024-64, Adam 5e-4)'
print('=== ' + name3 + ' ===')

m3 = Sequential([
    Dense(1024, activation='relu', input_dim=x_train_01.shape[1]),
    Dense(64,   activation='relu'),
    Dense(1,    activation='linear'),
])
m3.compile(optimizer=Adam(learning_rate=5e-4), loss='mse', metrics=['mae'])

rub3 = []
h3 = m3.fit(x_tr, y_tr,
            validation_data=(x_val, y_val),
            epochs=40, batch_size=512, verbose=0,
            callbacks=[make_rub_callback(m3, x_val, y_val_orig, rub3)])

mae3, _ = eval_net_scaled(m3, x_val, y_val_orig)
print(f'\nИтоговый MAE (тыс. руб.): {mae3:.2f}')
plot_history(h3, name3)

results.append({'name': name3,
                'params': 'Dense 1024-64, Adam 5e-4, bs=512, ep=40',
                'mae': mae3, 'history': h3, 'rub_history': rub3})
```

```
=== M3 wide (1024-64, Adam 5e-4) ===
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
  Эпоха   1 | MAE val (тыс. руб.) =  31.40
  Эпоха  10 | MAE val (тыс. руб.) =  30.94
  Эпоха  20 | MAE val (тыс. руб.) =  30.94
  Эпоха  30 | MAE val (тыс. руб.) =  30.96
  Эпоха  40 | MAE val (тыс. руб.) =  31.03

Итоговый MAE (тыс. руб.): 31.03
```

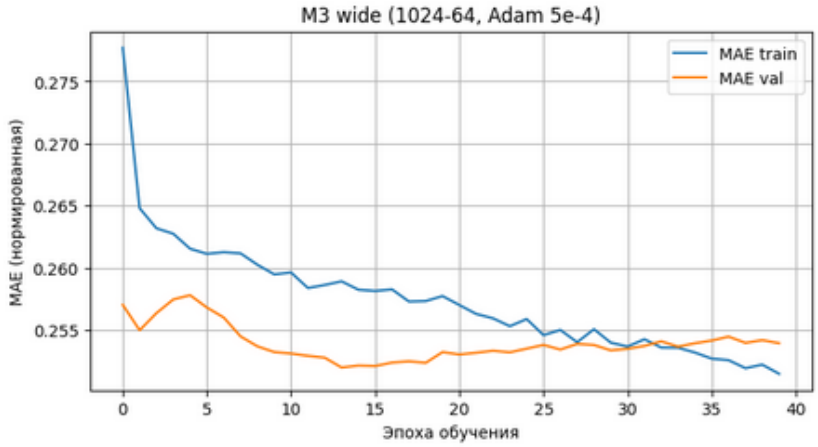


Рисунок 1.11 – Модель 3 Keras: обучение и кривые MAE. Итоговый MAE: 31.03 тыс. руб.

Модель 4 – Dropout-регуляризация.

Гиперпараметры: 3 слоя (256→128→64) с Dropout(0.3), Adam(lr=1e-3), batch_size=128, 60 эпох.

```python
name4 = 'M4 dropout (256-128-64 + Dropout 0.3)'
print('=== ' + name4 + ' ===')

m4 = Sequential([
    Dense(256, activation='relu', input_dim=x_train_01.shape[1]),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(64,  activation='relu'),
    Dropout(0.3),
    Dense(1,   activation='linear'),
])
m4.compile(optimizer=Adam(learning_rate=1e-3), loss='mse', metrics=['mae'])

rub4 = []
h4 = m4.fit(x_tr, y_tr,
            validation_data=(x_val, y_val),
            epochs=60, batch_size=128, verbose=0,
            callbacks=[make_rub_callback(m4, x_val, y_val_orig, rub4)])

mae4, _ = eval_net_scaled(m4, x_val, y_val_orig)
print(f'\nИтоговый MAE (тыс. руб.): {mae4:.2f}')
plot_history(h4, name4)

results.append({'name': name4,
                'params': 'Dense 256-128-64, Dropout 0.3, Adam 1e-3, bs=128, ep=60',
                'mae': mae4, 'history': h4, 'rub_history': rub4})
```

```
=== M4 dropout (256-128-64 + Dropout 0.3) ===
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
  Эпоха   1 | MAE val (тыс. руб.) =  34.32
  Эпоха  10 | MAE val (тыс. руб.) =  32.37
  Эпоха  20 | MAE val (тыс. руб.) =  32.48
  Эпоха  30 | MAE val (тыс. руб.) =  32.47
  Эпоха  40 | MAE val (тыс. руб.) =  32.41
  Эпоха  50 | MAE val (тыс. руб.) =  32.43
  Эпоха  60 | MAE val (тыс. руб.) =  32.37

Итоговый MAE (тыс. руб.): 32.37
```

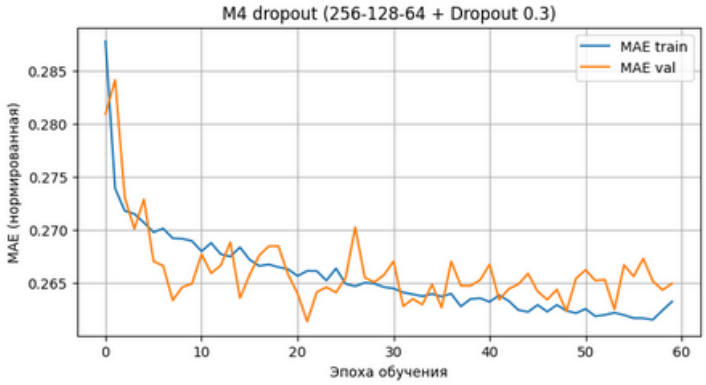


Рисунок 1.12 – Модель 4 Keras: обучение и кривые MAE. Итоговый MAE: 32.37 тыс. руб.

Модель 5 – Functional API + tanh + Adagrad.

Гиперпараметры: 3 слоя (200→tanh, 100→relu, 50→relu), Adagrad(lr=1e-2), batch_size=64, 40 эпох.

```python
name5 = 'M5 Functional+tanh (Adagrad 1e-2)'
print('=== ' + name5 + ' ===')

inp = Input(shape=(x_train_01.shape[1],))
x = Dense(200, activation='tanh')(inp)
x = Dense(100, activation='relu')(x)
x = Dense(50,  activation='relu')(x)
out = Dense(1, activation='linear')(x)
m5 = Model(inp, out)
m5.compile(optimizer=Adagrad(learning_rate=1e-2), loss='mse', metrics=['mae'])

rub5 = []
h5 = m5.fit(x_tr, y_tr,
            validation_data=(x_val, y_val),
            epochs=40, batch_size=64, verbose=0,
            callbacks=[make_rub_callback(m5, x_val, y_val_orig, rub5)])

mae5, _ = eval_net_scaled(m5, x_val, y_val_orig)
print(f'\nИтоговый MAE (тыс. руб.): {mae5:.2f}')
plot_history(h5, name5)

results.append({'name': name5,
                'params': 'Functional 200(tanh)-100-50, Adagrad 1e-2, bs=64, ep=40',
                'mae': mae5, 'history': h5, 'rub_history': rub5})
```

```
=== M5 Functional+tanh (Adagrad 1e-2) ===
  Эпоха   1 | MAE val (тыс. руб.) =  31.86
  Эпоха  10 | MAE val (тыс. руб.) =  31.62
  Эпоха  20 | MAE val (тыс. руб.) =  31.59
  Эпоха  30 | MAE val (тыс. руб.) =  31.55
  Эпоха  40 | MAE val (тыс. руб.) =  31.54

Итоговый MAE (тыс. руб.): 31.54
```

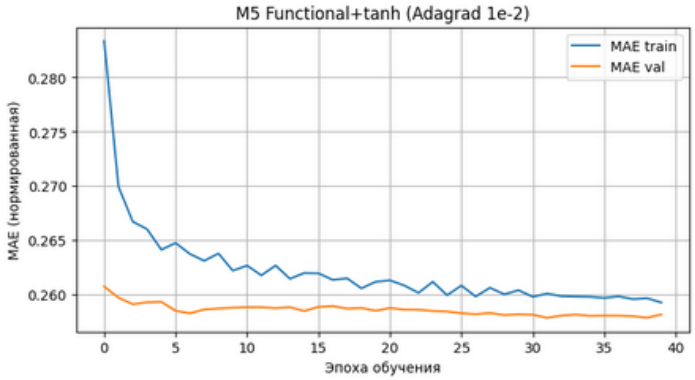


Рисунок 1.13 – Модель 5 Keras: обучение и кривые MAE. Итоговый MAE: 31.54 тыс. руб.

Модель 6 – пирамидальная глубокая (BN + Dropout, RMSprop).

Гиперпараметры: 5 слоёв (512→256→128→64→32), BN + Dropout(0.2), RMSprop(lr=5e-4), batch_size=256, 80 эпох.

```python
name6 = 'M6 deep pyramid (BN+Dropout, RMSprop 5e-4)'
print('=== ' + name6 + ' ===')

m6 = Sequential([
    Dense(512, input_dim=x_train_01.shape[1]),
    BatchNormalization(), Activation('relu'), Dropout(0.2),

    Dense(256), BatchNormalization(), Activation('relu'), Dropout(0.2),
    Dense(128), BatchNormalization(), Activation('relu'), Dropout(0.2),
    Dense(64),  BatchNormalization(), Activation('relu'),
    Dense(32),  Activation('relu'),
    Dense(1,    activation='linear'),
])
m6.compile(optimizer=RMSprop(learning_rate=5e-4), loss='mse', metrics=['mae'])

rub6 = []
h6 = m6.fit(x_tr, y_tr,
            validation_data=(x_val, y_val),
            epochs=80, batch_size=256, verbose=0,
            callbacks=[make_rub_callback(m6, x_val, y_val_orig, rub6)])

mae6, _ = eval_net_scaled(m6, x_val, y_val_orig)
print(f'\nИтоговый MAE (тыс. руб.): {mae6:.2f}')
plot_history(h6, name6)

results.append({'name': name6,
                'params': '5 Dense пирамида + BN + Dropout 0.2, RMSprop 5e-4, bs=256, ep=80',
                'mae': mae6, 'history': h6, 'rub_history': rub6})
```

```
=== M6 deep pyramid (BN+Dropout, RMSprop 5e-4) ===
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
  Эпоха   1 | MAE val (тыс. руб.) =  37.58
  Эпоха  10 | MAE val (тыс. руб.) =  30.95
  Эпоха  20 | MAE val (тыс. руб.) =  30.82
  Эпоха  30 | MAE val (тыс. руб.) =  30.82
  Эпоха  40 | MAE val (тыс. руб.) =  30.90
  Эпоха  50 | MAE val (тыс. руб.) =  30.82
  Эпоха  60 | MAE val (тыс. руб.) =  31.06
  Эпоха  70 | MAE val (тыс. руб.) =  30.88
  Эпоха  80 | MAE val (тыс. руб.) =  31.00

Итоговый MAE (тыс. руб.): 31.00
```

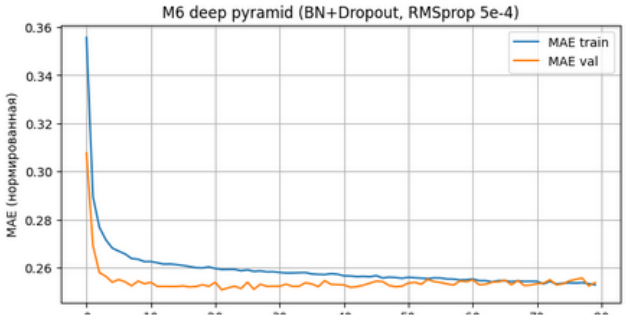


Рисунок 1.14 – Модель 6 Keras: обучение и кривые MAE. Итоговый MAE: 31.00 тыс. руб.

6. Сводный анализ результатов (Keras).

```python
# Таблица результатов
summary = pd.DataFrame([
    {'Модель': r['name'],
     'Гиперпараметры': r['params'],
     'MAE val (тыс. руб.)': round(r['mae'], 2)}
    for r in results
])
summary = summary.sort_values('MAE val (тыс. руб.)').reset_index(drop=True)
summary
```

	Модель	Гиперпараметры	MAE val (тыс. руб.)
0	M6 deep pyramid (BN+Dropout, RMSprop 5e-4)	5 Dense пирамида + BN + Dropout 0.2, RMSprop 5...	31.00
1	M3 wide (1024-64, Adam 5e-4)	Dense 1024-64, Adam 5e-4, bs=512, ep=40	31.03
2	M5 Functional+tanh (Adagrad 1e-2)	Functional 200(tanh)-100-50, Adagrad 1e-2, bs=...	31.54
3	M1 baseline (Dense 64-32, Adam 1e-3)	2×Dense (64,32), Adam 1e-3, bs=128, ep=30	31.62
4	M4 dropout (256-128-64 + Dropout 0.3)	Dense 256-128-64, Dropout 0.3, Adam 1e-3, bs=1...	32.37
5	M2 deep+BN (256-128-64-32, Adam 1e-3)	BN + 4 Dense (256,128,64,32), Adam 1e-3, bs=25...	32.49

Рисунок 1.15 – Сводная таблица результатов (Keras)

```
# График сравнения MAE (в тыс. руб.) по эпохам — все 6 моделей
plt.figure(figsize=(11, 6))
for r in results:
    plt.plot(r['rub_history'], label=r['name'])
plt.xlabel('Эпоха')
plt.ylabel('MAE val (тыс. руб.)')
plt.title('Сравнение моделей: MAE на валидации в тыс. руб.')
plt.legend(loc='upper right', fontsize=9)
plt.grid(); plt.show()
```

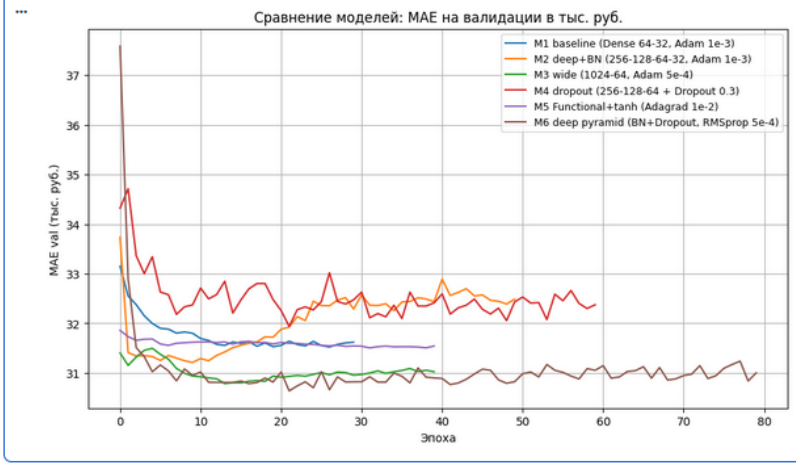


Рисунок 1.16 – Сравнение MAE на валидации по эпохам (Keras)

```
# Бар-чарт итоговых ошибок
plt.figure(figsize=(10, 5))
plt.bar(range(len(results)),
        [r['mae'] for r in results],
        color='steelblue')
plt.xticks(range(len(results)),
           [f'M{i+1}' for i in range(len(results))])
plt.ylabel('MAE val (тыс. руб.)')
plt.title('Итоговая ошибка каждой модели')
for i, r in enumerate(results):
    plt.text(i, r['mae'] + 0.3, f"{r['mae']:.1f}",
             ha='center', fontsize=10)
plt.grid(axis='y'); plt.show()
```

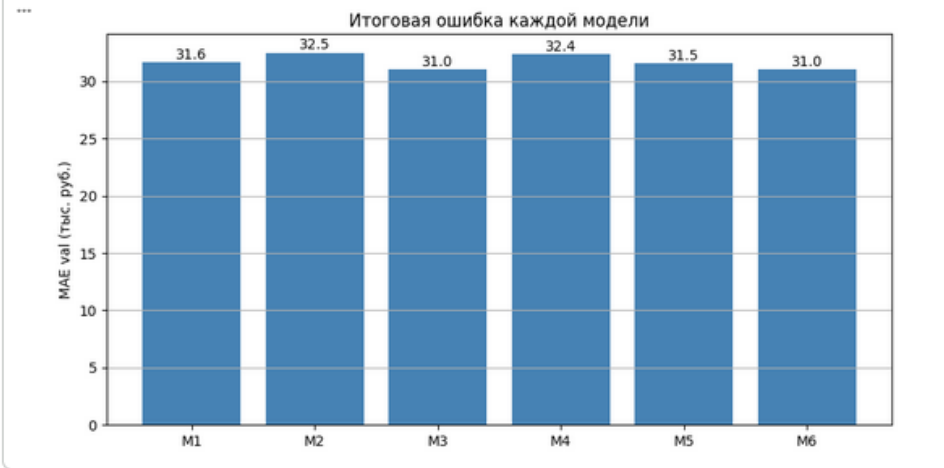


Рисунок 1.17 – Итоговая ошибка каждой Keras-модели (бар-чарт)

Задание 1.2. Реализация средствами библиотеки PyTorch.

1. Импорты.

Для реализации на PyTorch подключены: NumPy, Pandas, torch, torch.nn, TensorDataset и DataLoader для формирования батчей, StandardScaler и mean_absolute_error из scikit-learn, gdown, re и matplotlib. Зафиксирован seed 42

через np.random.seed, torch.manual_seed и cuda. Определено устройство (CUDA / CPU) – в ходе работы использовался GPU (cuda).

```python
# Работа с массивами и таблицами
import numpy as np
import pandas as pd

# PyTorch
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

# Прочее
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error
import gdown
import re
import matplotlib.pyplot as plt

%matplotlib inline

# Воспроизводимость
SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Используется устройство:', device)
```

```
Используется устройство: cuda
```

Рисунок 2.1 – Импорт библиотек (PyTorch), определение устройства

2. Подготовка данных.

Подготовка данных полностью повторяет раздел Keras-ноутбука: скачивается тот же CSV-файл с HeadHunter, выполняется очистка строк, парсинг столбцов и сборка числового вектора признаков длиной 39 элементов. Размер датафрейма: 62 967 строк × 12 столбцов.

```python
# Скачиваем базу
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l10/hh_fixed.csv',
               None, quiet=True)

# Чтение
df = pd.read_csv('hh_fixed.csv', index_col=0)
print('Размер исходного датафрейма:', df.shape)
df.head(3)
```

```
Размер исходного датафрейма: (62967, 12)
```

	Пол, возраст	ЗП	Ищет работу на должность:	Город	Занятость	График	Опыт (двойное нажатие для полной версии)	Последнее/нынешнее место работы	Последняя/нынешняя должность	Образование и ВУЗ	Обновление резюме	Авто
0	Мужчина , 29 лет , родился 16 мая 1989	40000 руб.	Специалист по поддержке чата(support team) дом...	Новороссийск , готов к переезду (Анапа, Геленд...	полная занятость	полный день	Опыт работы 3 года 9 месяцев Специалист по по...	ООО "Гольфстрим"	Генеральный директор	Высшее образование 2011 Международный юридиче...	26.04.2019 08:04	Не указано
1	Мужчина , 38 лет , родился 25 мая 1980	40000 руб.	Системный администратор	Новосибирск , м. Березовая роща , не готов к ...	полная занятость	полный день	Опыт работы 11 лет 11 месяцев Системный админ...	ООО «Завод модульных технологий»	Системный администратор	Высшее образование 2002 Новосибирский государс...	26.04.2019 04:30	Не указано
2	Мужчина , 35 лет , родился 14 июня 1983	300000 руб.	DevOps TeamLead / DevOps архитектор	Москва , готов к переезду , готов к редким ком...	полная занятость	полный день	Опыт работы 12 лет 11 месяцев DevOps TeamLead...	Банк ВТБ (ПАО)	Начальник отдела методологии разработки (DevOp...	DevOps TeamLead / DevOps архитектор 300 000 ру...	09.04.2019 14:40	Не указано

Рисунок 2.2 – Загрузка датасета и первые строки

```python
# Номера столбцов
COL_SEX_AGE  = df.columns.get_loc('Пол, возраст')
COL_SALARY   = df.columns.get_loc('ЗП')
COL_POS_SEEK = df.columns.get_loc('Ищет работу на должность:')
COL_POS_PREV = df.columns.get_loc('Последеняя/нынешняя должность')
COL_CITY     = df.columns.get_loc('Город')
COL_EMPL     = df.columns.get_loc('Занятость')
COL_SCHED    = df.columns.get_loc('График')
COL_EXP      = df.columns.get_loc('Опыт (двойное нажатие для полной версии)')
COL_EDU      = df.columns.get_loc('Образование и ВУЗ')
COL_UPDATED  = df.columns.get_loc('Обновление резюме')
```

```python
# Очистка строк
def purify(x):
    if isinstance(x, str):
        x = x.replace('\n', ' ').replace('\xa0', '').strip().lower()
    return x

# Применяем
for col in df.columns:
    df[col] = df[col].apply(purify)

# Извлечение года из даты
def extract_year(x):
    try:
        return int(re.search(r'\d\d.\d\d.(\d{4})', x)[1])
    except (IndexError, TypeError, ValueError):
        return 0
```

Рисунок 2.3 – Номера столбцов, очистка строк, извлечение года

```python
# Параметрические данные
currency_rate = {'usd': 65., 'kzt': 0.17, 'грн': 2.6, 'белруб': 30.5,
                 'eur': 70., 'kgs': 0.9, 'сум': 0.007, 'azn': 37.5}

age_class        = [0, [18, 23, 28, 33, 38, 43, 48, 53, 58, 63]]
experience_class = [0, [7, 13, 25, 37, 61, 97, 121, 157, 193, 241]]

city_class = [0,
              {'москва': 0, 'санкт-петербург': 1,
               'новосибирск': 2, 'екатеринбург': 2, 'нижний новгород': 2,
               'казань': 2, 'челябинск': 2, 'омск': 2, 'самара': 2,
               'ростов-на-дону': 2, 'уфа': 2, 'красноярск': 2, 'пермь': 2,
               'воронеж': 2, 'волгоград': 2,
               'прочие города': 3}]

employment_class = [0, {'стажировка': 0, 'частичная занятость': 1,
                        'проектная работа': 2, 'полная занятость': 3}]

schedule_class   = [0, {'гибкий график': 0, 'полный день': 1,
                        'сменный график': 2, 'удаленная работа': 3}]

education_class  = [0, {'высшее образование': 0, 'higher education': 0,
                        'среднее специальное': 1, 'неоконченное высшее': 2,
                        'среднее образование': 3}]

# Подсчёт количества классов
for class_lst in [age_class, experience_class]:
    class_lst[0] = len(class_lst[1]) + 1
for class_dct in [city_class, employment_class, schedule_class, education_class]:
    class_dct[0] = max(class_dct[1].values()) + 1

print('Возраст -', age_class[0], 'классов')
print('Опыт -', experience_class[0], 'классов')
print('Города -', city_class[0], 'классов')
```

```
Возраст - 11 классов
Опыт - 11 классов
Города - 4 классов
```

Рисунок 2.4 – Параметрические данные и подсчёт классов

```python
# Аналог keras.utils.to_categorical в numpy
def to_categorical(idx, num_classes):
    vec = np.zeros(num_classes, dtype=np.float32)
    vec[int(idx)] = 1.
    return vec

# One-hot для целочисленного значения
def int_to_ohe(arg, class_list):
    num_classes = class_list[0]
    for i in range(num_classes - 1):
        if arg < class_list[1][i]:
            cls = i
            break
    else:
        cls = num_classes - 1
    return to_categorical(cls, num_classes)

# Multi-hot из строки и словаря
def str_to_multi(arg, class_dict):
    num_classes = class_dict[0]
    result = np.zeros(num_classes, dtype=np.float32)
    for value, cls in class_dict[1].items():
        if value in arg:
            result[cls] = 1.
    return result
```

```python
# Парсинг пола и возраста
base_update_year = 2019

def extract_sex_age_years(arg):
    sex = 1. if 'муж' in arg else 0.
    try:
        years = base_update_year - int(re.search(r'\d{4}', arg)[0])
    except (IndexError, TypeError, ValueError):
        years = 0
    return sex, years

def age_years_to_ohe(arg):
    return int_to_ohe(arg, age_class)

def experience_months_to_ohe(arg):
    return int_to_ohe(arg, experience_class)

# Парсинг зарплаты
def extract_salary(arg):
    try:
        value = float(re.search(r'\d+', arg)[0])
        for currency, rate in currency_rate.items():
            if currency in arg:
                value *= rate
                break
    except TypeError:
        value = 0.
    return value / 1000.   # тыс. рублей
```

Рисунок 2.5 – Функции to_categorical, int_to_ohe, str_to_multi, парсинг пола/возраста/зарплаты

```python
# Город -> one-hot
def extract_city_to_ohe(arg):
    num_classes = city_class[0]
    split_array = re.split(r'[ ,.:()?!]', arg)
    for word in split_array:
        city_cls = city_class[1].get(word, -1)
        if city_cls >= 0:
            break
    else:
        city_cls = num_classes - 1
    return to_categorical(city_cls, num_classes)

# Multi-hot для занятости / графика / образования
def extract_employment_to_multi(arg):
    return str_to_multi(arg, employment_class)

def extract_schedule_to_multi(arg):
    return str_to_multi(arg, schedule_class)

def extract_education_to_multi(arg):
    result = str_to_multi(arg, education_class)
    if result[2] > 0.:          # неоконченное высшее => убираем «высшее»
        result[0] = 0.
    return result

# Опыт работы (месяцев)
def extract_experience_months(arg):
    try:
        years = int(re.search(r'(\d+)\s+(год.?|лет)', arg)[1])
    except (IndexError, TypeError, ValueError):
        years = 0
    try:
        months = int(re.search(r'(\d+)\s+месяц', arg)[1])
    except (IndexError, TypeError, ValueError):
        months = 0
    return years * 12 + months
```

```python
# Сборка одной строки
def extract_row_data(row):
    sex, age = extract_sex_age_years(row[COL_SEX_AGE])
    sex_vec   = np.array([sex], dtype=np.float32)
    age_ohe   = age_years_to_ohe(age)
    city_ohe  = extract_city_to_ohe(row[COL_CITY])
    empl_mult = extract_employment_to_multi(row[COL_EMPL])
    sched_mlt = extract_schedule_to_multi(row[COL_SCHED])
    edu_multi = extract_education_to_multi(row[COL_EDU])
    exp_mon   = extract_experience_months(row[COL_EXP])
    exp_ohe   = experience_months_to_ohe(exp_mon)
    salary    = extract_salary(row[COL_SALARY])
    salary_v  = np.array([salary], dtype=np.float32)

    x_data = np.hstack([sex_vec, age_ohe, city_ohe,
                        empl_mult, sched_mlt, edu_multi, exp_ohe])
    return x_data, salary_v

# Сбор всей выборки
def construct_train_data(row_list):
    x_data, y_data = [], []
    for row in row_list:
        x, y = extract_row_data(row)
        if y[0] > 0:
            x_data.append(x)
            y_data.append(y)
    return np.array(x_data, dtype=np.float32), np.array(y_data, dtype=np.float32)
```

Рисунок 2.6 – Функции извлечения города, занятости, графика, образования, опыта; сборка строки

```python
# Формируем выборку
x_train_01, y_train = construct_train_data(df.values)
print('x_train_01.shape =', x_train_01.shape)
print('y_train.shape    =', y_train.shape)
print('Пример x:', x_train_01[0])
print('Пример y:', y_train[0])
```

```
x_train_01.shape = (62967, 39)
y_train.shape    = (62967, 1)
Пример x: [1. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 1. 0. 1. 0. 0.
 1. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]
Пример y: [40.]
```

Рисунок 2.7 – Формирование выборки

3. Нормализация и DataLoader.

Целевая переменная нормирована StandardScaler. В отличие от Keras (где есть параметр validation_split), в PyTorch нет встроенного разделения, поэтому выборка делится вручную через маску (~15% валидации). Тензоры переносятся на устройство (CUDA). Формируются TensorDataset и DataLoader для train-выборки. INPUT_DIM = 39.

```
# Нормализация y
y_scaler = StandardScaler()
y_train_scaled = y_scaler.fit_transform(y_train).astype(np.float32)

print('mean =', round(y_train_scaled.mean(), 4),
      ', std =', round(y_train_scaled.std(), 4))

mean = -0.0 , std = 1.0
```

```
# Разделение по маске (~15% валидации)
np.random.seed(SEED)
val_mask = np.random.rand(len(x_train_01)) < 0.15
print('Train:', (~val_mask).sum(), '   Val:', val_mask.sum())

x_tr,  x_val  = x_train_01[~val_mask], x_train_01[val_mask]
y_tr,  y_val  = y_train_scaled[~val_mask], y_train_scaled[val_mask]
y_val_orig    = y_train[val_mask]  # для финальной денормированной MAE

# В тензоры
x_tr_t   = torch.from_numpy(x_tr).float()
y_tr_t   = torch.from_numpy(y_tr).float()
x_val_t  = torch.from_numpy(x_val).float().to(device)
y_val_t  = torch.from_numpy(y_val).float().to(device)

INPUT_DIM = x_train_01.shape[1]
print('INPUT_DIM =', INPUT_DIM)

Train: 53543    Val: 9424
INPUT_DIM = 39
```

Рисунок 2.8 – Нормализация y, разбивка по маске, перевод в тензоры, DataLoader

4. Универсальный обучающий цикл.

В PyTorch нет аналога model.fit(). Реализован универсальный цикл train_model: по каждому батчу выполняется forward → loss (MSELoss) → backward → optimizer.step(). В конце каждой эпохи через model.eval() вычисляется MAE на train и val (нормированная), затем denorm-MAE в тыс. руб. – аналог make_rub_callback из Keras.

```python
def train_model(model, optimizer, x_tr, y_tr, x_val_t, y_val_t,
                y_val_orig, y_scaler,
                epochs=30, batch_size=128, verbose_every=10):
    """Универсальный обучающий цикл с историей MAE (как Keras history)."""
    model = model.to(device)
    criterion = nn.MSELoss()

    # DataLoader для train
    train_ds = TensorDataset(x_tr, y_tr)
    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)

    history = {'mae': [], 'val_mae': []}    # нормированный MAE (как в Keras)
    rub_history = []                        # MAE в тыс. руб. на валидации

    for epoch in range(epochs):
        # ---- train ----
        model.train()
        for xb, yb in train_loader:
            xb, yb = xb.to(device), yb.to(device)
            optimizer.zero_grad()
            pred = model(xb)
            loss = criterion(pred, yb)
            loss.backward()
            optimizer.step()

        # ---- метрики на train и val (после эпохи) ----
        model.eval()
        with torch.no_grad():
            # train MAE (нормированный)
            preds_tr = []
            for xb, _ in DataLoader(train_ds, batch_size=1024):
                preds_tr.append(model(xb.to(device)).cpu().numpy())
            preds_tr = np.concatenate(preds_tr, axis=0)
            mae_tr = float(np.mean(np.abs(preds_tr - y_tr.numpy())))

            # val MAE (нормированный)
            preds_val = model(x_val_t).cpu().numpy()
            mae_val = float(np.mean(np.abs(preds_val - y_val_t.cpu().numpy())))

            # val MAE в тыс. руб.
            preds_val_rub = y_scaler.inverse_transform(preds_val)
            mae_rub = float(np.mean(np.abs(preds_val_rub - y_val_orig)))

        history['mae'].append(mae_tr)
        history['val_mae'].append(mae_val)
        rub_history.append(mae_rub)

        # Аналог Keras-callback-а: печать в конце эпохи
        if (epoch + 1) % verbose_every == 0 or epoch == 0:
            print(f'  Эпоха {epoch+1:3d}/{epochs} | '
                  f'MAE train={mae_tr:.4f}  val={mae_val:.4f}  | '
                  f'MAE val (тыс. руб.) = {mae_rub:6.2f}')

    return history, rub_history
```

Рисунок 2.9 – Универсальный обучающий цикл train_model

```python
def plot_history(history, title=''):
    plt.figure(figsize=(8, 4))
    plt.plot(history['mae'],     label='MAE train')
    plt.plot(history['val_mae'], label='MAE val')
    plt.title(title)
    plt.xlabel('Эпоха обучения')
    plt.ylabel('MAE (нормированная)')
    plt.legend(); plt.grid(); plt.show()

def eval_net_scaled(model, x_val_t, y_val_orig, y_scaler):
    model.eval()
    with torch.no_grad():
        preds = model(x_val_t).cpu().numpy()
    preds = y_scaler.inverse_transform(preds)
    return mean_absolute_error(y_val_orig, preds), preds
```

```python
# Здесь будут результаты всех 6 моделей
results = []  # [{name, params, mae, history, rub_history}]
```

Рисунок 2.10 – Вспомогательные функции plot_history и eval_net_scaled (PyTorch)

5. Обучение и сравнение шести моделей (PyTorch).

Модель 1.

Гиперпараметры: nn.Sequential с 2×Linear (64, 32), активация ReLU, Adam(lr=1e-3), batch_size=128, 30 эпох.

```
name1 = 'M1 baseline (Linear 64-32, Adam 1e-3)'
print('=== ' + name1 + ' ===')

torch.manual_seed(SEED)
m1 = nn.Sequential(
    nn.Linear(INPUT_DIM, 64), nn.ReLU(),
    nn.Linear(64, 32),        nn.ReLU(),
    nn.Linear(32, 1),
)
opt1 = torch.optim.Adam(m1.parameters(), lr=1e-3)

h1, rub1 = train_model(m1, opt1, x_tr_t, y_tr_t, x_val_t, y_val_t,
                       y_val_orig, y_scaler,
                       epochs=30, batch_size=128, verbose_every=5)

mae1, _ = eval_net_scaled(m1, x_val_t, y_val_orig, y_scaler)
print(f'\nИтоговый MAE (тыс. руб.): {mae1:.2f}')
plot_history(h1, name1)

results.append({'name': name1,
                'params': '2×Linear (64,32), Adam 1e-3, bs=128, ep=30',
                'mae': mae1, 'history': h1, 'rub_history': rub1})
```

```
=== M1 baseline (Linear 64-32, Adam 1e-3) ===
  Эпоха   1/30 | MAE train=0.2679  val=0.2664  | MAE val (тыс. руб.) =  32.55
  Эпоха   5/30 | MAE train=0.2608  val=0.2597  | MAE val (тыс. руб.) =  31.73
  Эпоха  10/30 | MAE train=0.2547  val=0.2539  | MAE val (тыс. руб.) =  31.02
  Эпоха  15/30 | MAE train=0.2574  val=0.2574  | MAE val (тыс. руб.) =  31.45
  Эпоха  20/30 | MAE train=0.2598  val=0.2600  | MAE val (тыс. руб.) =  31.77
  Эпоха  25/30 | MAE train=0.2575  val=0.2579  | MAE val (тыс. руб.) =  31.51
  Эпоха  30/30 | MAE train=0.2538  val=0.2546  | MAE val (тыс. руб.) =  31.11

Итоговый MAE (тыс. руб.): 31.11
```

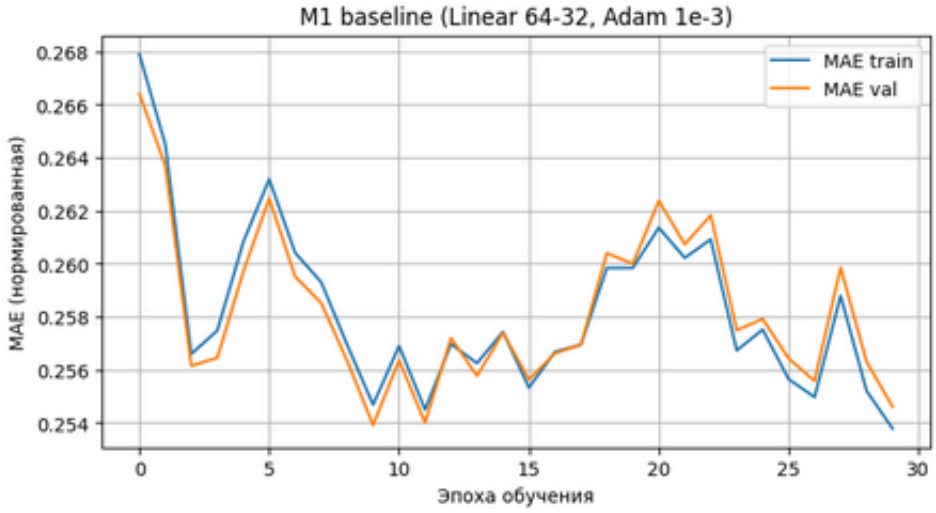


Рисунок 2.11 – Модель 1 PyTorch: обучение и кривые MAE. Итоговый MAE: 31.11 тыс. руб.

Модель 2 – глубокая с BatchNorm1d.

Гиперпараметры: 4 скрытых слоя (256→128→64→32), BatchNorm1d после входа, Adam(lr=1e-3), batch_size=256, 50 эпох.

```python
name2 = 'M2 deep+BN (256-128-64-32, Adam 1e-3)'
print('=== ' + name2 + ' ===')

torch.manual_seed(SEED)
m2 = nn.Sequential(
    nn.BatchNorm1d(INPUT_DIM),
    nn.Linear(INPUT_DIM, 256), nn.ReLU(),
    nn.Linear(256, 128),       nn.ReLU(),
    nn.Linear(128, 64),        nn.ReLU(),
    nn.Linear(64, 32),         nn.ReLU(),
    nn.Linear(32, 1),
)
opt2 = torch.optim.Adam(m2.parameters(), lr=1e-3)

h2, rub2 = train_model(m2, opt2, x_tr_t, y_tr_t, x_val_t, y_val_t,
                       y_val_orig, y_scaler,
                       epochs=50, batch_size=256, verbose_every=10)

mae2, _ = eval_net_scaled(m2, x_val_t, y_val_orig, y_scaler)
print(f'\nИтоговый MAE (тыс. руб.): {mae2:.2f}')
plot_history(h2, name2)

results.append({'name': name2,
                'params': 'BN + 4 Linear (256,128,64,32), Adam 1e-3, bs=256, ep=50',
                'mae': mae2, 'history': h2, 'rub_history': rub2})
```

```
=== M2 deep+BN (256-128-64-32, Adam 1e-3) ===
  Эпоха   1/50 | MAE train=0.2642  val=0.2644  | MAE val (тыс. руб.) =  32.30
  Эпоха  10/50 | MAE train=0.2542  val=0.2548  | MAE val (тыс. руб.) =  31.13
  Эпоха  20/50 | MAE train=0.2575  val=0.2619  | MAE val (тыс. руб.) =  31.99
  Эпоха  30/50 | MAE train=0.2506  val=0.2580  | MAE val (тыс. руб.) =  31.52
  Эпоха  40/50 | MAE train=0.2462  val=0.2578  | MAE val (тыс. руб.) =  31.49
  Эпоха  50/50 | MAE train=0.2416  val=0.2569  | MAE val (тыс. руб.) =  31.39

Итоговый MAE (тыс. руб.): 31.39
```

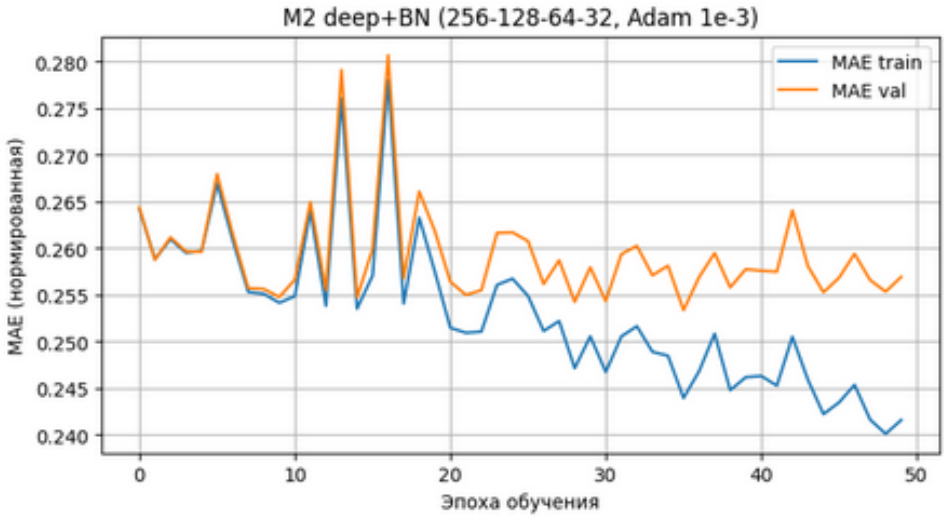


Рисунок 2.12 – Модель 2 PyTorch: обучение и кривые MAE. Итоговый MAE: 31.39 тыс. руб.

Модель 3 – широкая (1 большой скрытый слой).

Гиперпараметры: Linear(1024) + ReLU → Linear(64), Adam(lr=5e-4), batch_size=512, 40 эпох.

```python
name3 = 'M3 wide (1024-64, Adam 5e-4)'
print('=== ' + name3 + ' ===')

torch.manual_seed(SEED)
m3 = nn.Sequential(
    nn.Linear(INPUT_DIM, 1024), nn.ReLU(),
    nn.Linear(1024, 64),        nn.ReLU(),
    nn.Linear(64, 1),
)
opt3 = torch.optim.Adam(m3.parameters(), lr=5e-4)

h3, rub3 = train_model(m3, opt3, x_tr_t, y_tr_t, x_val_t, y_val_t,
                       y_val_orig, y_scaler,
                       epochs=40, batch_size=512, verbose_every=10)

mae3, _ = eval_net_scaled(m3, x_val_t, y_val_orig, y_scaler)
print(f'\nИтоговый MAE (тыс. руб.): {mae3:.2f}')
plot_history(h3, name3)

results.append({'name': name3,
                'params': 'Linear 1024-64, Adam 5e-4, bs=512, ep=40',
                'mae': mae3, 'history': h3, 'rub_history': rub3})
```

```
=== M3 wide (1024-64, Adam 5e-4) ===
  Эпоха   1/40 | MAE train=0.2566  val=0.2553  | MAE val (тыс. руб.) =  31.19
  Эпоха  10/40 | MAE train=0.2551  val=0.2551  | MAE val (тыс. руб.) =  31.17
  Эпоха  20/40 | MAE train=0.2592  val=0.2607  | MAE val (тыс. руб.) =  31.86
  Эпоха  30/40 | MAE train=0.2798  val=0.2826  | MAE val (тыс. руб.) =  34.53
  Эпоха  40/40 | MAE train=0.2558  val=0.2604  | MAE val (тыс. руб.) =  31.82

Итоговый MAE (тыс. руб.): 31.82
```

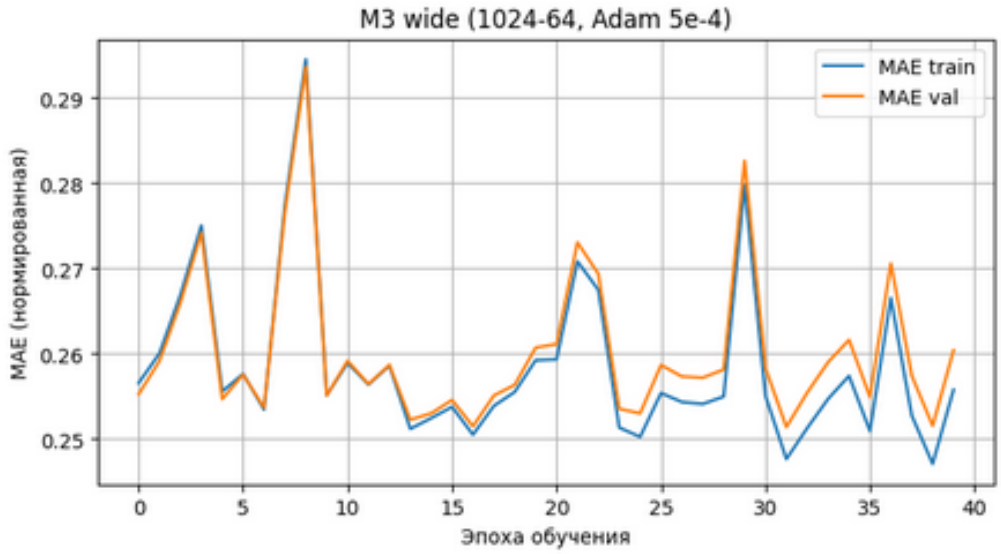


Рисунок 2.13 – Модель 3 PyTorch: обучение и кривые MAE. Итоговый MAE: 31.82 тыс. руб.

Модель 4 – Dropout-регуляризация.

Гиперпараметры: 3 слоя (256→128→64) с nn.Dropout(0.3), Adam(lr=1e-3), batch_size=128, 60 эпох.

```python
name4 = 'M4 dropout (256-128-64 + Dropout 0.3)'
print('=== ' + name4 + ' ===')

torch.manual_seed(SEED)
m4 = nn.Sequential(
    nn.Linear(INPUT_DIM, 256), nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(256, 128),       nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(128, 64),        nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(64, 1),
)
opt4 = torch.optim.Adam(m4.parameters(), lr=1e-3)

h4, rub4 = train_model(m4, opt4, x_tr_t, y_tr_t, x_val_t, y_val_t,
                       y_val_orig, y_scaler,
                       epochs=60, batch_size=128, verbose_every=10)

mae4, _ = eval_net_scaled(m4, x_val_t, y_val_orig, y_scaler)
print(f'\nИтоговый MAE (тыс. руб.): {mae4:.2f}')
plot_history(h4, name4)

results.append({'name': name4,
                'params': 'Linear 256-128-64, Dropout 0.3, Adam 1e-3, bs=128, ep=60',
                'mae': mae4, 'history': h4, 'rub_history': rub4})
```

```
=== M4 dropout (256-128-64 + Dropout 0.3) ===
  Эпоха   1/60 | MAE train=0.2616  val=0.2605  | MAE val (тыс. руб.) =  31.83
  Эпоха  10/60 | MAE train=0.2862  val=0.2855  | MAE val (тыс. руб.) =  34.89
  Эпоха  20/60 | MAE train=0.2568  val=0.2585  | MAE val (тыс. руб.) =  31.58
  Эпоха  30/60 | MAE train=0.2561  val=0.2600  | MAE val (тыс. руб.) =  31.76
  Эпоха  40/60 | MAE train=0.2530  val=0.2568  | MAE val (тыс. руб.) =  31.38
  Эпоха  50/60 | MAE train=0.2507  val=0.2555  | MAE val (тыс. руб.) =  31.22
  Эпоха  60/60 | MAE train=0.2558  val=0.2618  | MAE val (тыс. руб.) =  31.99

Итоговый MAE (тыс. руб.): 31.99
```

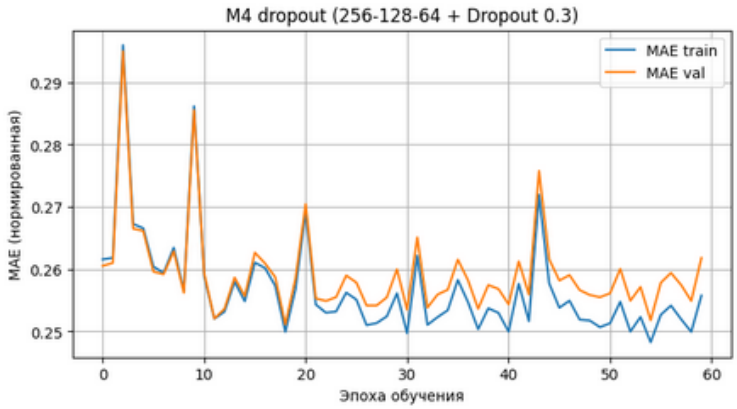


Рисунок 2.14 – Модель 4 PyTorch: обучение и кривые MAE. Итоговый MAE: 31.99 тыс. руб.

Модель 5 – Tanh + Adagrad.

Гиперпараметры: 3 слоя (200→Tanh, 100→ReLU, 50→ReLU), Adagrad(lr=1e-2), batch_size=64, 40 эпох. Модель перемещена на устройство до инициализации оптимизатора.

```python
name5 = 'M5 Tanh+Adagrad (200-100-50, Adagrad 1e-2)'
print('=== ' + name5 + ' ===')

torch.manual_seed(SEED)
m5 = nn.Sequential(
    nn.Linear(INPUT_DIM, 200), nn.Tanh(),
    nn.Linear(200, 100),       nn.ReLU(),
    nn.Linear(100, 50),        nn.ReLU(),
    nn.Linear(50, 1),
)
m5.to(device) # Moved model to device before initializing optimizer
opt5 = torch.optim.Adagrad(m5.parameters(), lr=1e-2)

h5, rub5 = train_model(m5, opt5, x_tr_t, y_tr_t, x_val_t, y_val_t,
                       y_val_orig, y_scaler,
                       epochs=40, batch_size=64, verbose_every=10)

mae5, _ = eval_net_scaled(m5, x_val_t, y_val_orig, y_scaler)
print(f'\nИтоговый MAE (тыс. руб.): {mae5:.2f}')
plot_history(h5, name5)

results.append({'name': name5,
                'params': '200(Tanh)-100-50, Adagrad 1e-2, bs=64, ep=40',
                'mae': mae5, 'history': h5, 'rub_history': rub5})
```

```
=== M5 Tanh+Adagrad (200-100-50, Adagrad 1e-2) ===
  Эпоха   1/40 | MAE train=0.2623  val=0.2612  | MAE val (тыс. руб.) =  31.91
  Эпоха  10/40 | MAE train=0.2573  val=0.2568  | MAE val (тыс. руб.) =  31.37
  Эпоха  20/40 | MAE train=0.2574  val=0.2577  | MAE val (тыс. руб.) =  31.49
  Эпоха  30/40 | MAE train=0.2569  val=0.2583  | MAE val (тыс. руб.) =  31.56
  Эпоха  40/40 | MAE train=0.2544  val=0.2564  | MAE val (тыс. руб.) =  31.33

Итоговый MAE (тыс. руб.): 31.33
```

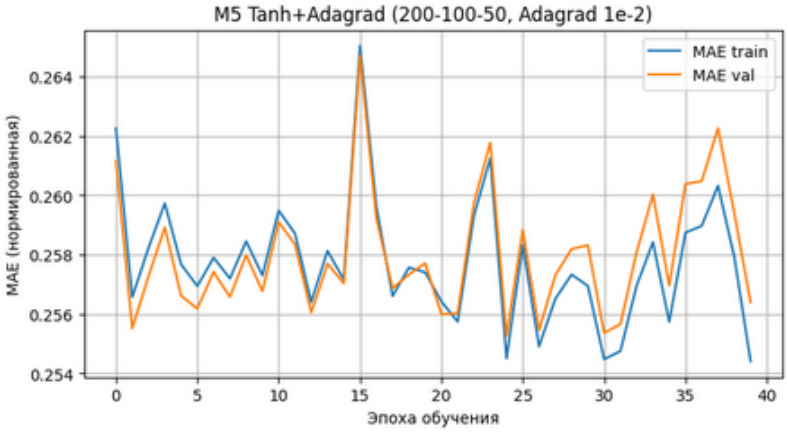


Рисунок 2.15 – Модель 5 PyTorch: обучение и кривые MAE. Итоговый MAE: 31.33 тыс. руб.

Модель 6 – пирамидальная глубокая (BN + Dropout, RMSprop).

Гиперпараметры: 5 слоёв (512→256→128→64→32), BatchNorm1d + Dropout(0.2) между слоями, RMSprop(lr=5e-4), batch_size=256, 80 эпох.

```python
name6 = 'M6 deep pyramid (BN+Dropout, RMSprop 5e-4)'
print('=== ' + name6 + ' ===')

torch.manual_seed(SEED)
m6 = nn.Sequential(
    nn.Linear(INPUT_DIM, 512), nn.BatchNorm1d(512), nn.ReLU(), nn.Dropout(0.2),
    nn.Linear(512, 256),       nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.2),
    nn.Linear(256, 128),       nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(0.2),
    nn.Linear(128, 64),        nn.BatchNorm1d(64),  nn.ReLU(),
    nn.Linear(64, 32),                              nn.ReLU(),
    nn.Linear(32, 1),
)
opt6 = torch.optim.RMSprop(m6.parameters(), lr=5e-4)

h6, rub6 = train_model(m6, opt6, x_tr_t, y_tr_t, x_val_t, y_val_t,
                       y_val_orig, y_scaler,
                       epochs=80, batch_size=256, verbose_every=10)

mae6, _ = eval_net_scaled(m6, x_val_t, y_val_orig, y_scaler)
print(f'\nИтоговый MAE (тыс. руб.): {mae6:.2f}')
plot_history(h6, name6)

results.append({'name': name6,
                'params': '5 Linear пирамида + BN + Dropout 0.2, RMSprop 5e-4, bs=256, ep=80',
                'mae': mae6, 'history': h6, 'rub_history': rub6})
```

```
=== M6 deep pyramid (BN+Dropout, RMSprop 5e-4) ===
  Эпоха   1/80 | MAE train=0.2565  val=0.2558  | MAE val (тыс. руб.) =  31.25
  Эпоха  10/80 | MAE train=0.2569  val=0.2588  | MAE val (тыс. руб.) =  31.62
  Эпоха  20/80 | MAE train=0.2482  val=0.2508  | MAE val (тыс. руб.) =  30.64
  Эпоха  30/80 | MAE train=0.2527  val=0.2562  | MAE val (тыс. руб.) =  31.31
  Эпоха  40/80 | MAE train=0.2465  val=0.2519  | MAE val (тыс. руб.) =  30.77
  Эпоха  50/80 | MAE train=0.2493  val=0.2557  | MAE val (тыс. руб.) =  31.24
  Эпоха  60/80 | MAE train=0.2524  val=0.2601  | MAE val (тыс. руб.) =  31.78
  Эпоха  70/80 | MAE train=0.2470  val=0.2551  | MAE val (тыс. руб.) =  31.17
  Эпоха  80/80 | MAE train=0.2443  val=0.2546  | MAE val (тыс. руб.) =  31.11

Итоговый MAE (тыс. руб.): 31.11
```

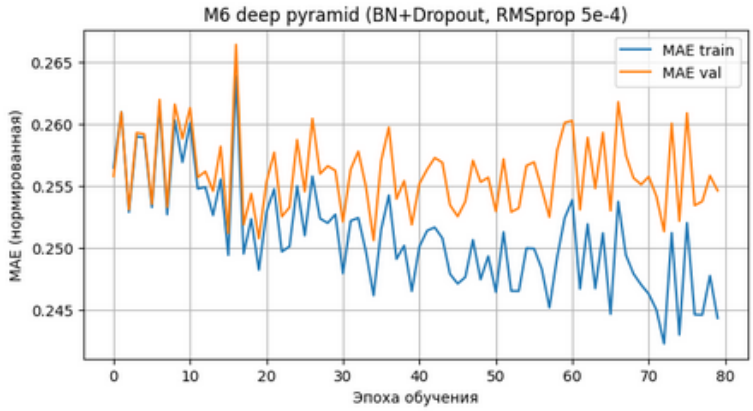


Рисунок 2.16 – Модель 6 PyTorch: обучение и кривые MAE. Итоговый MAE: 31.11 тыс. руб.

6. Сводный анализ результатов (PyTorch).

```python
# Таблица результатов
summary = pd.DataFrame([
    {'Модель': r['name'],
     'Гиперпараметры': r['params'],
     'MAE val (тыс. руб.)': round(r['mae'], 2)}
    for r in results
])
summary = summary.sort_values('MAE val (тыс. руб.)').reset_index(drop=True)
summary
```

	Модель	Гиперпараметры	MAE val (тыс. руб.)
0	M1 baseline (Linear 64-32, Adam 1e-3)	2×Linear (64,32), Adam 1e-3, bs=128, ep=30	31.11
1	M6 deep pyramid (BN+Dropout, RMSprop 5e-4)	5 Linear пирамида + BN + Dropout 0.2, RMSprop ...	31.11
2	M5 Tanh+Adagrad (200-100-50, Adagrad 1e-2)	200(Tanh)-100-50, Adagrad 1e-2, bs=64, ep=40	31.33
3	M2 deep+BN (256-128-64-32, Adam 1e-3)	BN + 4 Linear (256,128,64,32), Adam 1e-3, bs=2...	31.39
4	M3 wide (1024-64, Adam 5e-4)	Linear 1024-64, Adam 5e-4, bs=512, ep=40	31.82
5	M4 dropout (256-128-64 + Dropout 0.3)	Linear 256-128-64, Dropout 0.3, Adam 1e-3, bs=...	31.99

Рисунок 2.17 – Сводная таблица результатов (PyTorch)

```python
# График сравнения MAE (в тыс. руб.) по эпохам — все 6 моделей
plt.figure(figsize=(11, 6))
for r in results:
    plt.plot(r['rub_history'], label=r['name'])
plt.xlabel('Эпоха')
plt.ylabel('MAE val (тыс. руб.)')
plt.title('Сравнение моделей: MAE на валидации в тыс. руб.')
plt.legend(loc='upper right', fontsize=9)
plt.grid(); plt.show()
```

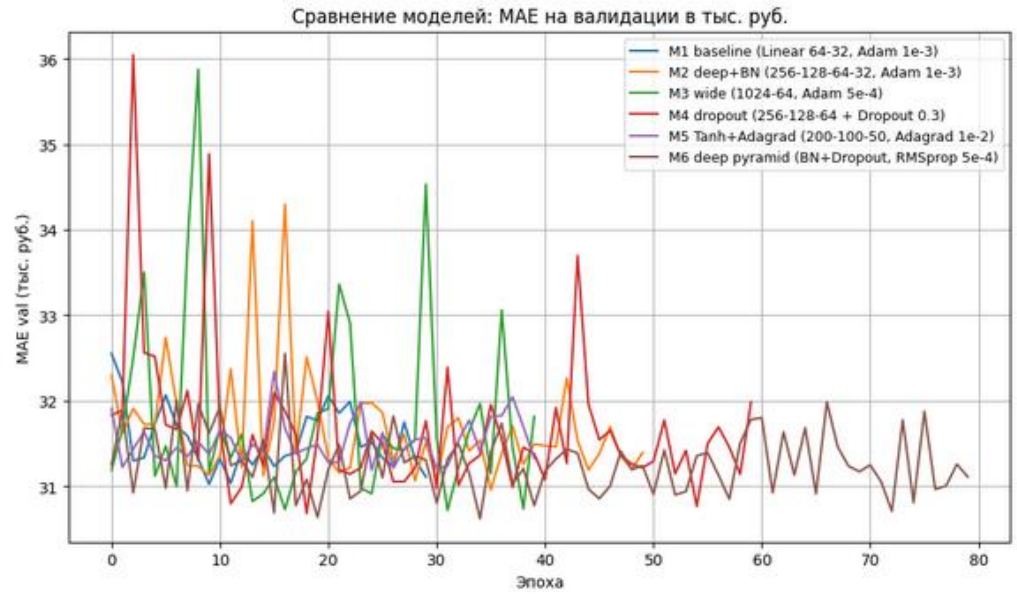


Рисунок 2.18 – Сравнение MAE на валидации по эпохам (PyTorch)

```python
# Бар-чарт итоговых ошибок
plt.figure(figsize=(10, 5))
plt.bar(range(len(results)),
        [r['mae'] for r in results],
        color='steelblue')
plt.xticks(range(len(results)),
           [f'M{i+1}' for i in range(len(results))])
plt.ylabel('MAE val (тыс. руб.)')
plt.title('Итоговая ошибка каждой модели')
for i, r in enumerate(results):
    plt.text(i, r['mae'] + 0.3, f"{r['mae']:.1f}",
             ha='center', fontsize=10)
plt.grid(axis='y'); plt.show()
```

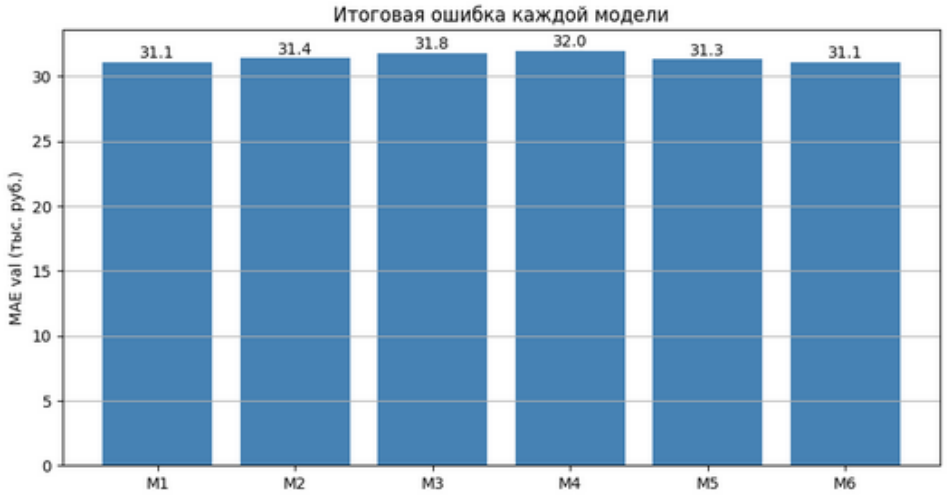


Рисунок 2.19 – Итоговая ошибка каждой PyTorch-модели (бар-чарт)

Задание 2. Предсказание очков в баскетбольных матчах.

1. Импорты.

Для решения задачи подключены: NumPy, Pandas, TensorFlow/Keras (Model, Functional API, Dense, Dropout, BatchNormalization, Input, concatenate), оптимизатор Adam, коллбэки LambdaCallback и ModelCheckpoint, Tokenizer для токенизации текста, StandardScaler и matplotlib. Зафиксирован seed 42.

```
# Работа с массивами и таблицами
import numpy as np
import pandas as pd

# TensorFlow / Keras
import tensorflow as tf
from tensorflow.keras import utils
from tensorflow.keras.models import Model
from tensorflow.keras.layers import (Dense, Dropout, BatchNormalization,
                                     Input, concatenate)
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import LambdaCallback, ModelCheckpoint
from tensorflow.keras.preprocessing.text import Tokenizer

# Прочее
from sklearn.preprocessing import StandardScaler
import gdown
import matplotlib.pyplot as plt

%matplotlib inline

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

Рисунок 3.1 – Импорт библиотек

2. Подготовка данных.

Датасет basketball.csv (52 450 строк, 9 столбцов) загружен с облачного хранилища. Текстовая колонка info токенизируется Tokenizer с максимальным словарём 5 000 слов и преобразуется в матрицу Bag-of-Words (BoW). Числовые признаки (Ком. 1, Ком. 2, Минута, Секунда, ftime) собираются в матрицу xTrain (52 450 × 5). Целевая переменная – fcount (количество очков). Итоговые размерности: xTrain – (52450, 5), yTrain – (52450,), xBOW_text – (52450, 5000).

```python
# Загрузка из google облака
gdown.download(
    'https://storage.yandexcloud.net/aiueducation/Content/base/l10/basketball.csv',
    None, quiet=True
)

df = pd.read_csv('basketball.csv', encoding='cp1251', sep=';',
                 header=0, index_col=0)
print('Размер датафрейма:', df.shape)
df.head()
```

```
Размер датафрейма: (52450, 9)
```

	TOTAL	info	Ком. 1	Ком. 2	Минута	Общая минута	Секунда	fcount	ftime
0	98,5	4081445 Новая Зеландия. Женщины. WBC. Регулярн...	2	0.0	1	1.0	30	81	90.0
1	100,5	4081445 Новая Зеландия. Женщины. WBC. Регулярн...	2	2.0	1	1.0	45	81	105.0
2	99,5	4081445 Новая Зеландия. Женщины. WBC. Регулярн...	2	2.0	2	2.0	0	81	120.0
3	98,5	4081445 Новая Зеландия. Женщины. WBC. Регулярн...	2	2.0	2	2.0	30	81	150.0
4	95,5	4081445 Новая Зеландия. Женщины. WBC. Регулярн...	2	2.0	3	3.0	0	81	180.0

```python
# Текстовые данные
data_text = df['info'].values
print('Длина текстового списка:', len(data_text))

# Токенизатор
maxWordsCount = 5000
sim_for_del   = '!"#$%&()*+,-./:;<=>?@[\\]^_`{|}~\t\n'

tokenizer = Tokenizer(
    num_words=maxWordsCount,
    filters=sim_for_del,
    lower=True,
    split=' ',
    oov_token='unknown',
    char_level=False
)
tokenizer.fit_on_texts(data_text)
```

```
Длина текстового списка: 52450
```

```python
# Эмбеддинг-последовательности и Bag of Words
Sequences  = tokenizer.texts_to_sequences(data_text)
xBOW_text  = tokenizer.sequences_to_matrix(Sequences)

# Числовые признаки
xTrain = np.array(df[['Ком. 1', 'Ком. 2', 'Минута', 'Секунда', 'ftime']]
                  .astype('int'))
yTrain = np.array(df['fcount'].astype('int'))

print('xTrain.shape    =', xTrain.shape)
print('yTrain.shape    =', yTrain.shape)
print('xBOW_text.shape =', xBOW_text.shape)
```

```
xTrain.shape    = (52450, 5)
yTrain.shape    = (52450,)
xBOW_text.shape = (52450, 5000)
```

Рисунок 3.2 – Загрузка датасета, токенизация текста, формирование числовых признаков и BoW

3. Нормализация и train/val split.

Числовые признаки нормированы StandardScaler (5 признаков с разными масштабами – особенно «секунды» против «минут»). Целевая переменная тоже нормирована – это ускоряет сходимость; после предсказания денормируется для подсчёта MAE в очках. BoW-матрица бинарная [0,1] и не нормируется. Разбивка на train/val маской ~15%: Train – 44 567, Val – 7 883. NUM_DIM = 5, TEXT_DIM = 5000.

```python
# Скейлинг
x_scaler = StandardScaler()
xTrain_scaled = x_scaler.fit_transform(xTrain).astype(np.float32)

y_scaler = StandardScaler()
yTrain_scaled = y_scaler.fit_transform(yTrain.reshape(-1, 1)).astype(np.float32)

xBOW_text = xBOW_text.astype(np.float32)

# Маска валидации
np.random.seed(SEED)
val_mask = np.random.rand(len(xTrain)) < 0.15
print('Train:', (~val_mask).sum(), '   Val:', val_mask.sum())

x_tr_num,  x_val_num  = xTrain_scaled[~val_mask],  xTrain_scaled[val_mask]
x_tr_txt,  x_val_txt  = xBOW_text[~val_mask],      xBOW_text[val_mask]
y_tr_sc,   y_val_sc   = yTrain_scaled[~val_mask],  yTrain_scaled[val_mask]
y_tr_orig, y_val_orig = yTrain[~val_mask],         yTrain[val_mask]

NUM_DIM  = xTrain_scaled.shape[1]    # 5
TEXT_DIM = xBOW_text.shape[1]        # 5000
print('NUM_DIM =', NUM_DIM, ', TEXT_DIM =', TEXT_DIM)
```

```
Train: 44567    Val: 7883
NUM_DIM = 5 , TEXT_DIM = 5000
```

Рисунок 3.3 – Нормализация признаков и целевой переменной, формирование train/val масок

4. Модель с двумя входами (Functional API).

Построена модель с двумя ветками. Числовая ветка: 5 → BatchNorm → Dense(32) → Dense(32), relu. Текстовая ветка (BoW, 5000): Dropout(0.3) → Dense(256) → Dropout(0.3) → Dense(64), relu. Ветки конкатенируются: Dense(32, relu) → Dense(1, linear). Текстовая ветка несёт основной сигнал (BoW высокоразмерный), числовая корректирует. Dropout на текстовой ветке и BatchNormalization на числовой – стандартные регуляризаторы для подобной задачи. Итого: 1 301 109 параметров.

```python
# Числовая ветка
input_num = Input(shape=(NUM_DIM,), name='num_input')
x1 = BatchNormalization()(input_num)
x1 = Dense(32, activation='relu')(x1)
x1 = Dense(32, activation='relu')(x1)

# Текстовая ветка
input_txt = Input(shape=(TEXT_DIM,), name='txt_input')
x2 = Dropout(0.3)(input_txt)
x2 = Dense(256, activation='relu')(x2)
x2 = Dropout(0.3)(x2)
x2 = Dense(64, activation='relu')(x2)

# Объединение
x = concatenate([x1, x2])
x = Dense(32, activation='relu')(x)
out = Dense(1, activation='linear')(x)

model = Model([input_num, input_txt], out)
model.compile(optimizer=Adam(learning_rate=1e-3), loss='mse', metrics=['mae'])
model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
txt_input (InputLayer)	(None, 5000)	0	-
num_input (InputLayer)	(None, 5)	0	-
dropout (Dropout)	(None, 5000)	0	txt_input[0][0]
batch_normalization (BatchNormalizatio…	(None, 5)	20	num_input[0][0]
dense_2 (Dense)	(None, 256)	1,280,256	dropout[0][0]
dense (Dense)	(None, 32)	192	batch_normalizat…
dropout_1 (Dropout)	(None, 256)	0	dense_2[0][0]
dense_1 (Dense)	(None, 32)	1,056	dense[0][0]
dense_3 (Dense)	(None, 64)	16,448	dropout_1[0][0]
concatenate (Concatenate)	(None, 96)	0	dense_1[0][0], dense_3[0][0]
dense_4 (Dense)	(None, 32)	3,104	concatenate[0][0]
dense_5 (Dense)	(None, 1)	33	dense_4[0][0]

Total params: 1,301,109 (4.96 MB)
Trainable params: 1,301,099 (4.96 MB)
Non-trainable params: 10 (40.00 B)

Рисунок 3.4 – Архитектура модели с двумя входами (Functional API) и её summary

5. Обучение с callback.

Реализован callback on_epoch_end_callback (аналог шаблонного check_MAE_predict1_DubbleInput): после каждой эпохи делает predict на val, денормирует y_scaler.inverse_transform, считает MAE в очках и печатает результат. Параллельно ModelCheckpoint сохраняет лучшие веса по val_loss. Обучение: 50 эпох, batch_size=512, Adam(lr=1e-3). После обучения загружаются лучшие веса – финальный MAE на валидации составил 0.844 очка.

```python
# История MAE в очках (для графика)
mae_pts_history = []

def on_epoch_end_callback(epoch, logs):
    pred_sc = model.predict([x_val_num, x_val_txt], verbose=0)
    pred_pts = y_scaler.inverse_transform(pred_sc).ravel()
    mae_pts = float(np.mean(np.abs(pred_pts - y_val_orig)))
    mae_pts_history.append(mae_pts)
    star = '  ★ цель достигнута' if mae_pts <= 17.0 else ''
    print(f'  Эпоха {epoch+1:3d} | '
          f'loss={logs.get("loss"):.4f}  val_loss={logs.get("val_loss"):.4f}  | '
          f'MAE val (очки) = {mae_pts:6.3f}{star}')

mae_callback = LambdaCallback(on_epoch_end=on_epoch_end_callback)

# Чекпоинт лучшей модели
checkpoint = ModelCheckpoint(
    'best_model.weights.h5',
    monitor='val_loss',
    save_best_only=True,
    save_weights_only=True,
    verbose=0,
)
```

Рисунок 3.5 – Callback для вычисления MAE в очках и ModelCheckpoint

```python
history = model.fit(
    [x_tr_num, x_tr_txt], y_tr_sc,
    validation_data=([x_val_num, x_val_txt], y_val_sc),
    epochs=50,
    batch_size=512,
    verbose=0,
    callbacks=[mae_callback, checkpoint],
)

# Загружаем веса лучшей эпохи
model.load_weights('best_model.weights.h5')

# Финальная MAE на валидации с лучшими весами
pred_sc  = model.predict([x_val_num, x_val_txt], verbose=0)
pred_pts = y_scaler.inverse_transform(pred_sc).ravel()
final_mae = float(np.mean(np.abs(pred_pts - y_val_orig)))
print(f'\nЛучшая модель — MAE на валидации: {final_mae:.3f} очков')
```

```
...  Эпоха  1 | loss=0.3121  val_loss=0.0397  | MAE val (очки) =  3.057  ★ цель достигнута
     Эпоха  2 | loss=0.0811  val_loss=0.0224  | MAE val (очки) =  2.952  ★ цель достигнута
     Эпоха  3 | loss=0.0571  val_loss=0.0167  | MAE val (очки) =  2.556  ★ цель достигнута
     Эпоха  4 | loss=0.0497  val_loss=0.0148  | MAE val (очки) =  2.329  ★ цель достигнута
     Эпоха  5 | loss=0.0424  val_loss=0.0173  | MAE val (очки) =  2.614  ★ цель достигнута
     Эпоха  6 | loss=0.0388  val_loss=0.0157  | MAE val (очки) =  2.407  ★ цель достигнута
     Эпоха  7 | loss=0.0382  val_loss=0.0153  | MAE val (очки) =  2.309  ★ цель достигнута
     Эпоха  8 | loss=0.0358  val_loss=0.0092  | MAE val (очки) =  1.811  ★ цель достигнута
     Эпоха  9 | loss=0.0314  val_loss=0.0132  | MAE val (очки) =  2.062  ★ цель достигнута
     Эпоха 10 | loss=0.0323  val_loss=0.0120  | MAE val (очки) =  2.019  ★ цель достигнута
     Эпоха 11 | loss=0.0281  val_loss=0.0099  | MAE val (очки) =  1.725  ★ цель достигнута
     Эпоха 12 | loss=0.0274  val_loss=0.0128  | MAE val (очки) =  2.015  ★ цель достигнута
     Эпоха 13 | loss=0.0275  val_loss=0.0110  | MAE val (очки) =  1.802  ★ цель достигнута
     Эпоха 14 | loss=0.0271  val_loss=0.0122  | MAE val (очки) =  1.829  ★ цель достигнута
     Эпоха 15 | loss=0.0262  val_loss=0.0092  | MAE val (очки) =  1.623  ★ цель достигнута
     Эпоха 16 | loss=0.0240  val_loss=0.0102  | MAE val (очки) =  1.671  ★ цель достигнута
     Эпоха 17 | loss=0.0226  val_loss=0.0082  | MAE val (очки) =  1.587  ★ цель достигнута
     Эпоха 18 | loss=0.0228  val_loss=0.0082  | MAE val (очки) =  1.494  ★ цель достигнута
     Эпоха 19 | loss=0.0218  val_loss=0.0091  | MAE val (очки) =  1.544  ★ цель достигнута
     Эпоха 20 | loss=0.0218  val_loss=0.0090  | MAE val (очки) =  1.463  ★ цель достигнута
     Эпоха 21 | loss=0.0204  val_loss=0.0060  | MAE val (очки) =  1.174  ★ цель достигнута
     Эпоха 22 | loss=0.0202  val_loss=0.0096  | MAE val (очки) =  1.607  ★ цель достигнута
     Эпоха 23 | loss=0.0223  val_loss=0.0070  | MAE val (очки) =  1.252  ★ цель достигнута
     Эпоха 24 | loss=0.0202  val_loss=0.0058  | MAE val (очки) =  1.096  ★ цель достигнута
     Эпоха 25 | loss=0.0185  val_loss=0.0111  | MAE val (очки) =  1.774  ★ цель достигнута
     Эпоха 26 | loss=0.0225  val_loss=0.0068  | MAE val (очки) =  1.054  ★ цель достигнута
     Эпоха 27 | loss=0.0186  val_loss=0.0097  | MAE val (очки) =  1.432  ★ цель достигнута
     Эпоха 28 | loss=0.0195  val_loss=0.0072  | MAE val (очки) =  1.239  ★ цель достигнута
     Эпоха 29 | loss=0.0182  val_loss=0.0073  | MAE val (очки) =  1.290  ★ цель достигнута
     Эпоха 30 | loss=0.0157  val_loss=0.0073  | MAE val (очки) =  1.206  ★ цель достигнута
     Эпоха 31 | loss=0.0176  val_loss=0.0032  | MAE val (очки) =  0.844  ★ цель достигнута
     Эпоха 32 | loss=0.0169  val_loss=0.0059  | MAE val (очки) =  1.071  ★ цель достигнута
     Эпоха 33 | loss=0.0167  val_loss=0.0055  | MAE val (очки) =  0.956  ★ цель достигнута
     Эпоха 34 | loss=0.0181  val_loss=0.0069  | MAE val (очки) =  1.284  ★ цель достигнута
     Эпоха 35 | loss=0.0144  val_loss=0.0053  | MAE val (очки) =  0.969  ★ цель достигнута
     Эпоха 36 | loss=0.0170  val_loss=0.0032  | MAE val (очки) =  0.771  ★ цель достигнута
     Эпоха 37 | loss=0.0163  val_loss=0.0060  | MAE val (очки) =  1.069  ★ цель достигнута
     Эпоха 38 | loss=0.0153  val_loss=0.0062  | MAE val (очки) =  1.004  ★ цель достигнута
     Эпоха 39 | loss=0.0150  val_loss=0.0058  | MAE val (очки) =  0.995  ★ цель достигнута
     Эпоха 40 | loss=0.0149  val_loss=0.0063  | MAE val (очки) =  1.153  ★ цель достигнута
     Эпоха 41 | loss=0.0175  val_loss=0.0036  | MAE val (очки) =  0.890  ★ цель достигнута
     Эпоха 42 | loss=0.0152  val_loss=0.0040  | MAE val (очки) =  0.802  ★ цель достигнута
     Эпоха 43 | loss=0.0136  val_loss=0.0050  | MAE val (очки) =  0.880  ★ цель достигнута
     Эпоха 44 | loss=0.0154  val_loss=0.0066  | MAE val (очки) =  1.163  ★ цель достигнута
     Эпоха 45 | loss=0.0128  val_loss=0.0064  | MAE val (очки) =  1.009  ★ цель достигнута
     Эпоха 46 | loss=0.0158  val_loss=0.0070  | MAE val (очки) =  1.165  ★ цель достигнута
     Эпоха 47 | loss=0.0130  val_loss=0.0051  | MAE val (очки) =  0.898  ★ цель достигнута
     Эпоха 48 | loss=0.0137  val_loss=0.0070  | MAE val (очки) =  1.018  ★ цель достигнута
     Эпоха 49 | loss=0.0150  val_loss=0.0061  | MAE val (очки) =  0.909  ★ цель достигнута
     Эпоха 50 | loss=0.0148  val_loss=0.0046  | MAE val (очки) =  0.931  ★ цель достигнута

     Лучшая модель — MAE на валидации: 0.844 очков
```

Рисунок 3.6 – Лог обучения

6. График обучения.

График обучения отображает две оси: слева – MSE (loss и val_loss в нормированных единицах), справа – MAE в очках на валидации. Зелёная пунктирная линия – целевое значение MAE = 17 очков. Модель уже на первых эпохах опустилась значительно ниже цели и продолжала улучшаться, достигнув менее 1 очка MAE.

```
fig, ax1 = plt.subplots(figsize=(10, 5))

ax1.plot(history.history['loss'],     'b-', label='train_loss (MSE на нормированных)')
ax1.plot(history.history['val_loss'], 'b--', label='val_loss')
ax1.set_xlabel('Эпоха')
ax1.set_ylabel('loss', color='b')
ax1.tick_params(axis='y', labelcolor='b')
ax1.grid()

ax2 = ax1.twinx()
ax2.plot(mae_pts_history, 'r-', label='MAE val (очки)')
ax2.axhline(17, color='g', linestyle='--', label='Цель: MAE = 17')
ax2.set_ylabel('MAE val (очки)', color='r')
ax2.tick_params(axis='y', labelcolor='r')

# Объединяем легенды
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper right')

plt.title('Обучение модели с двумя входами')
plt.show()
```

Рисунок 3.7 – График обучения: MSE и MAE val (очки) по эпохам. Цель MAE ≤ 17 достигнута

7. Финальная проверка.

Функция check_MAE_predict1_DubbleInput повторяет логику шаблона: делает predict, денормирует, считает MAE и строит scatter «правильные значения vs предсказания». При plot=True рисуется scatter с диагональной линией идеального предсказания.

```python
def check_MAE_predict1_DubbleInput(model, x_data, x_data_text,
                                   y_data_not_scaled, plot=False):
    pred_sc = model.predict([x_data, x_data_text], verbose=0)
    y_pred  = y_scaler.inverse_transform(pred_sc).ravel()

    mae = float(np.mean(np.abs(y_data_not_scaled - y_pred)))
    rel = mae / y_data_not_scaled.mean() * 100
    n   = len(y_data_not_scaled)
    print(f'Средняя абсолютная ошибка {mae:.3f} очков '
          f'это {rel:.3f}% от общей выборки в {n} игр')

    if plot:
        plt.figure(figsize=(7, 7))
        plt.scatter(y_data_not_scaled, y_pred, alpha=0.3, s=8)
        plt.xlabel('Правильные значения (очки)')
        plt.ylabel('Предсказания (очки)')
        plt.axis('equal')
        lim = (0, max(y_data_not_scaled.max(), y_pred.max()) + 10)
        plt.xlim(lim); plt.ylim(lim)
        plt.plot([0, lim[1]], [0, lim[1]], 'r--')
        plt.title(f'MAE = {mae:.2f} очков')
        plt.grid()
        plt.show()
    return mae
```

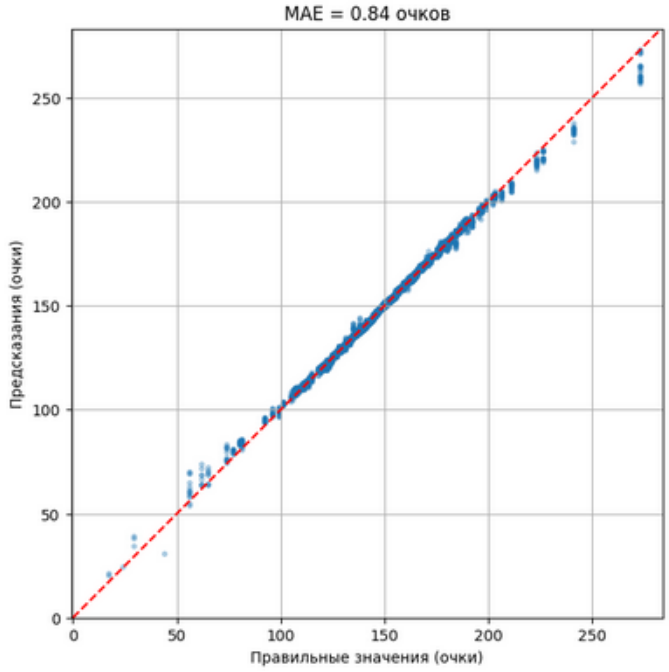
Рисунок 3.8 – Функция финальной проверки check_MAE_predict1_DubbleInput

Результаты финальной проверки: на валидационной выборке (7 883 игр) MAE = 0.844 очка – это 0.558% от общей выборки; на обучающей выборке (44 567 игр) MAE = 0.829 очка – 0.546%. Scatter-диаграмма подтверждает высокое качество предсказания: точки плотно группируются вдоль диагонали.

```python
# Финальная проверка на валидации
print('--- Валидация ---')
mae_val = check_MAE_predict1_DubbleInput(
    model, x_val_num, x_val_txt, y_val_orig, plot=True
)

# Контрольная проверка на train
print('\n--- Train ---')
mae_tr = check_MAE_predict1_DubbleInput(
    model, x_tr_num, x_tr_txt, y_tr_orig, plot=False
)
```

```
--- Валидация ---
WARNING:matplotlib.axes._base:Ignoring fixed x limits to fulfill fixed data aspect with adjustable data limits.
Средняя абсолютная ошибка 0.844 очков это 0.558% от общей выборки в 7883 игр
```




```
--- Train ---
Средняя абсолютная ошибка 0.829 очков это 0.546% от общей выборки в 44567 игр
```

Рисунок 3.9 – Scatter предсказания vs реальные очки (val). MAE = 0.84 очка

Итог: целевое значение MAE ≤ 17 очков достигнуто с большим запасом – финальная MAE на валидации составила 0.844 очка (менее 1 очка), что на порядок лучше поставленной цели. Это достигнуто за счёт двухвходовой архитектуры Functional API, которая совместно использует текстовый BoW-сигнал и числовые признаки матча.

Задание 3. Предсказание цен на автомобили.

1. Импорты.

Подключены: NumPy, Pandas, TensorFlow/Keras (Sequential, Dense, Adam, ModelCheckpoint, EarlyStopping), gdown и matplotlib. Seed зафиксирован равным 42.

```
import numpy as np
import pandas as pd

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping

import gdown
import matplotlib.pyplot as plt
%matplotlib inline

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

Рисунок 4.1 – Импорт библиотек для задания 3

2. Загрузка и осмотр данных.

Датасет cars_new.csv загружен с облачного хранилища (70 119 строк, 10 столбцов: mark, model, price, year, mileage, body, kpp, fuel, volume, power). Числовые столбцы: price, year, mileage, volume, power; категориальные: mark, model, body, kpp, fuel. Пропуски: body – 3, kpp – 4, fuel – 1. Диапазон цен до очистки: 16 000 – 24 500 000 руб., средняя цена – 527 726 руб.

```
gdown.download(
    'https://storage.yandexcloud.net/aiueducation/Content/base/l10/cars_new.csv',
    None, quiet=True
)

try:
    df = pd.read_csv('cars_new.csv')
except UnicodeDecodeError:
    df = pd.read_csv('cars_new.csv', encoding='cp1251')

print('Размер:', df.shape)
df.head()
```

```
Размер: (70119, 10)
```

	mark	model	price	year	mileage	body	kpp	fuel	volume	power
0	kia	cerato	996000	2018	28000	седан	автомат	бензин	2.0	150.0
1	daewoo	nexia 1 поколение [2-й рестайлинг]	140200	2012	60500	седан	механика	бензин	1.5	80.0
2	suzuki	jimny 3 поколение [рестайлинг]	750000	2011	29000	внедорожник	автомат	бензин	1.3	85.0
3	bmw	x1 18 e84 [рестайлинг]	970000	2014	49500	кроссовер	автомат	бензин	2.0	150.0
4	chevrolet	lacetti 1 поколение	205000	2007	151445	седан	механика	бензин	1.4	95.0

```
print('--- dtypes ---')
print(df.dtypes)
print('\n--- Пропуски ---')
print(df.isna().sum())
```

```
--- dtypes ---
mark        object
model       object
price        int64
year         int64
mileage      int64
body        object
kpp         object
fuel        object
volume     float64
power      float64
dtype: object

--- Пропуски ---
mark       0
model      0
price      0
year       0
mileage    0
body       3
kpp        4
fuel       0
volume     0
power      0
dtype: int64
```

Рисунок 4.2 – Загрузка датасета, первые строки, типы данных и пропуски

3. Подготовка данных.

Ключевое решение – жёсткая фильтрация диапазона цен: убраны объявления дешевле 50 000 руб. и дороже 7 000 000 руб. (мусорные объявления по 1/100/11 111 руб. и суперкары). После очистки осталось 68 587 строк (отсеяно 1 532). Средняя цена: 531 536 руб., диапазон: 50 000 – 6 990 000 руб.

Редкие категории в cat_cols схлопываются в «__other__» (MIN_FREQ=10), затем применяется pd.get_dummies. Числовые столбцы нормируются делением на максимум. Итоговая форма X: (68587, 1160).

```python
# Подбираем имя целевого столбца
target_col = None
for cand in ['price', 'Price', 'цена', 'Цена', 'cost', 'value']:
    if cand in df.columns:
        target_col = cand
        break
if target_col is None:
    numeric = df.select_dtypes(include=[np.number])
    target_col = numeric.mean().idxmax()

print(f'Целевой столбец: {target_col!r}')
print(f'До очистки: {len(df)} строк, ср.цена {df[target_col].mean():,.0f}')
print(f'Мин/Макс: {df[target_col].min():,.0f} / {df[target_col].max():,.0f}')
```

```
Целевой столбец: 'price'
До очистки: 70119 строк, ср.цена 527,726
Мин/Макс: 16,000 / 24,500,000
```

```python
# Жёсткая очистка цены — это решает проблему большого % ошибки
data = df.dropna().copy()

PRICE_MIN = 50_000      # ниже — мусорные объявления
PRICE_MAX = 7_000_000   # выше — суперкары, выбросы

before = len(data)
data = data[(data[target_col] >= PRICE_MIN) & (data[target_col] <= PRICE_MAX)]
print(f'После очистки цены: {len(data)} строк (отсеяли {before - len(data)})')
print(f'Средняя цена: {data[target_col].mean():,.0f}')
print(f'Мин/Макс: {data[target_col].min():,.0f} / {data[target_col].max():,.0f}')
```

```
После очистки цены: 68587 строк (отсеяли 1525)
Средняя цена: 531,536
Мин/Макс: 50,000 / 6,990,000
```

```python
# Признаки и target
y_full = data[target_col].values.astype(np.float32)
X_full = data.drop(columns=[target_col]).copy()

num_cols = X_full.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = X_full.select_dtypes(exclude=[np.number]).columns.tolist()

print('Числовые:    ', num_cols)
print('Категориальные:', cat_cols)
```

```
Числовые:     ['year', 'mileage', 'volume', 'power']
Категориальные: ['mark', 'model', 'body', 'kpp', 'fuel']
```

Рисунок 4.3 – Очистка цен, определение признаков и целевой переменной

```python
# Схлопываем редкие категории — иначе one-hot раздувается на сотни столбцов
MIN_FREQ = 10
for c in cat_cols:
    counts = X_full[c].value_counts()
    rare = counts[counts < MIN_FREQ].index
    if len(rare):
        X_full.loc[X_full[c].isin(rare), c] = '__other__'
    print(f'  {c}: уникальных {X_full[c].nunique()} (было {len(counts)})')

# One-hot
X_encoded = pd.get_dummies(X_full, columns=cat_cols, drop_first=False).astype(np.float32)

# Нормируем числовые столбцы делением на максимум (как в эталоне курса)
x_max = {}
for c in num_cols:
    m = float(X_encoded[c].abs().max()) or 1.0
    X_encoded[c] = X_encoded[c] / m
    x_max[c] = m

print('\nИтоговая форма X:', X_encoded.shape)
```

```
  mark: уникальных 21 (было 21)
  model: уникальных 1113 (было 3098)
  body: уникальных 13 (было 16)
  kpp: уникальных 4 (было 4)
  fuel: уникальных 5 (было 5)

Итоговая форма X: (68587, 1160)
```

Рисунок 4.4 – Схлопывание редких категорий, One-Hot Encoding, нормировка числовых столбцов

4. Разбиение на 3 выборки: train / val / test (70 / 15 / 15).

Случайная перестановка индексов (np.random.default_rng(SEED)), разбивка 70/15/15: Train – 48 010 строк, Val – 10 288, Test – 10 289. Целевая переменная нормируется делением на Y_MAX = максимум train-цен (6 990 000 руб.) – это позволяет после предсказания умножить обратно на Y_MAX для денормировки.

```
rng = np.random.default_rng(SEED)
n   = len(X_encoded)
idx = rng.permutation(n)

n_tr  = int(n * 0.70)
n_val = int(n * 0.15)

idx_tr   = idx[:n_tr]
idx_val  = idx[n_tr:n_tr + n_val]
idx_test = idx[n_tr + n_val:]

X_tr,   y_tr   = X_encoded.values[idx_tr],   y_full[idx_tr]
X_val,  y_val  = X_encoded.values[idx_val],  y_full[idx_val]
X_test, y_test = X_encoded.values[idx_test], y_full[idx_test]

# Нормировка цены: делим на максимум train. После предсказания умножим обратно.
Y_MAX = float(y_tr.max())
y_tr_n   = (y_tr   / Y_MAX).astype(np.float32)
y_val_n  = (y_val  / Y_MAX).astype(np.float32)
y_test_n = (y_test / Y_MAX).astype(np.float32)

print('Train:', X_tr.shape, 'Val:', X_val.shape, 'Test:', X_test.shape)
print(f'Y_MAX (train) = {Y_MAX:,.0f}')
```

```
Train: (48010, 1160) Val: (10288, 1160) Test: (10289, 1160)
Y_MAX (train) = 6,990,000
```

Рисунок 4.5 – Разбиение на train/val/test (70/15/15), нормировка целевой переменной

5. Модель.

Простая полносвязная сеть Sequential: Dense(200, relu, input_dim=1160) → Dense(200, relu) → Dense(1, linear). Без Dropout и BatchNorm – на данной задаче они только мешают. Оптимизатор Adam(lr=1e-3), функция потерь MSE. Итого: 272 601 обучаемый параметр.

```
INPUT_DIM = X_tr.shape[1]

model = Sequential([
    Dense(200, activation='relu', input_dim=INPUT_DIM),
    Dense(200, activation='relu'),
    Dense(1,   activation='linear'),
])
model.compile(optimizer=Adam(1e-3), loss='mse')
model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `inp
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 200)	232,200
dense_1 (Dense)	(None, 200)	40,200
dense_2 (Dense)	(None, 1)	201

```
Total params: 272,601 (1.04 MB)
Trainable params: 272,601 (1.04 MB)
Non-trainable params: 0 (0.00 B)
```

Рисунок 4.6 – Архитектура модели Sequential и её summary

6. Обучение.

Обучение до 200 эпох с batch_size=128, ModelCheckpoint (сохраняет лучшие веса по val_loss) и EarlyStopping (patience=20). Остановка на эпохе 112 (лучшая – эпоха 92), лучший val_loss = 0.000502. Кривые MSE показывают равномерное снижение без переобучения.

```
checkpoint = ModelCheckpoint('best_cars.weights.h5', monitor='val_loss',
                             save_best_only=True, save_weights_only=True, verbose=0)
early_stop = EarlyStopping(monitor='val_loss', patience=20, restore_best_weights=True, verbose=1)

history = model.fit(
    X_tr, y_tr_n,
    validation_data=(X_val, y_val_n),
    epochs=200,
    batch_size=128,
    verbose=0,
    callbacks=[checkpoint, early_stop],
)
print(f'Лучший val_loss: {min(history.history["val_loss"]):.6f}')
```

```
Epoch 112: early stopping
Restoring model weights from the end of the best epoch: 92.
Лучший val_loss: 0.000502
```

```
plt.figure(figsize=(10, 5))
plt.plot(history.history['loss'],     label='train MSE')
plt.plot(history.history['val_loss'], label='val MSE')
plt.xlabel('Эпоха'); plt.ylabel('MSE (норм.)')
plt.title('История обучения'); plt.legend(); plt.grid(); plt.show()
```

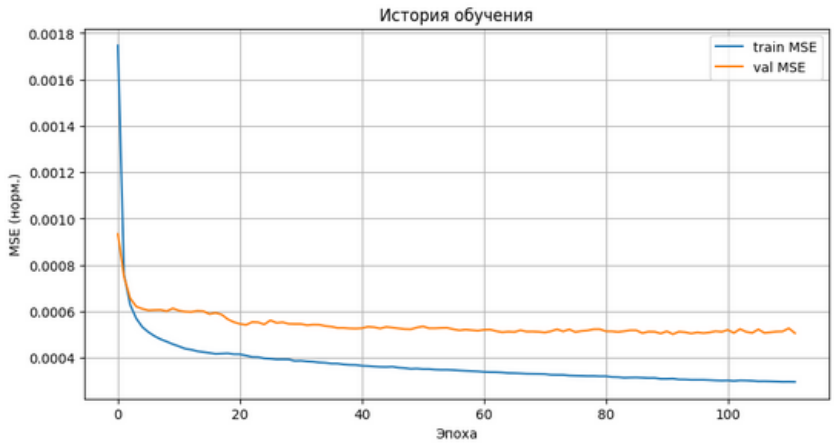


Рисунок 4.7 – Код обучения, лог early stopping и кривые MSE (train/val)

7. Средний процент ошибки на проверочной выборке.

Предсказание денормируется умножением на Y_MAX. Результаты на val-выборке: MAE = 84 712 руб., средняя цена val = 535 294 руб., процент ошибки = 15.83%. Scatter-диаграмма «реальная цена vs предсказание» показывает хорошую линейную зависимость вдоль диагонали.

```
# Предсказание + обратная нормировка (умножаем обратно на Y_MAX)
pred_val = (model.predict(X_val, verbose=0).ravel() * Y_MAX)

mae_val      = float(np.mean(np.abs(pred_val - y_val)))
mean_price_v = float(y_val.mean())
err_pct_val  = mae_val / mean_price_v * 100

print(f'MAE val          : {mae_val:>12,.0f} руб.')
print(f'Средняя цена val : {mean_price_v:>12,.0f} руб.')
print(f'Процент ошибки   : {err_pct_val:>11.2f}%')
```

```
MAE val          :       84,712 руб.
Средняя цена val :      535,294 руб.
Процент ошибки   :       15.83%
```

```
plt.figure(figsize=(7, 7))
plt.scatter(y_val, pred_val, alpha=0.3, s=8)
lim = (0, max(y_val.max(), pred_val.max()) * 1.05)
plt.plot(lim, lim, 'r--')
plt.xlim(lim); plt.ylim(lim)
plt.xlabel('Реальная цена'); plt.ylabel('Предсказание')
plt.title(f'Val: MAE = {mae_val:,.0f} ({err_pct_val:.2f}%)')
plt.grid(); plt.show()
```

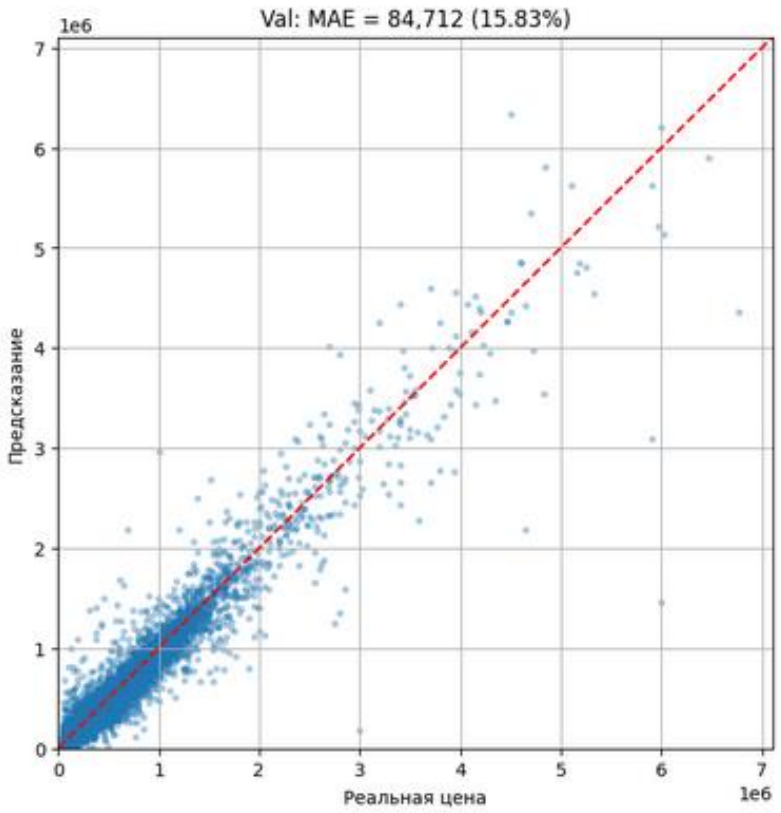


Рисунок 4.8 – Предсказание на val, денормировка, MAE и scatter

8. Ошибка на каждом примере тестовой выборки.

На тестовой выборке (10 289 примеров): MAE test = 87 730 руб., средняя цена test = 536 994 руб., суммарный процент ошибки = 16.34%. Таблица первых 20 примеров содержит реальную цену, предсказание, абсолютную ошибку в рублях и процент ошибки на каждом примере.

```python
pred_test = (model.predict(X_test, verbose=0).ravel() * Y_MAX)

errors_abs = np.abs(pred_test - y_test)        # модуль ошибки в рублях на каждом примере
errors_pct = errors_abs / y_test * 100         # % ошибки на каждом примере

mae_test     = float(errors_abs.mean())
mean_price_t = float(y_test.mean())
err_pct_test = mae_test / mean_price_t * 100   # ← это и есть «суммарный процент ошибки» из задания

print(f'Размер test               : {len(y_test)}')
print(f'MAE test                  : {mae_test:>12,.0f} руб.')
print(f'Средняя цена test         : {mean_price_t:>12,.0f} руб.')
print(f'СУММАРНЫЙ ПРОЦЕНТ ОШИБКИ  : {err_pct_test:>11.2f}%')
```

```
Размер test               : 10289
MAE test                  :       87,730 руб.
Средняя цена test         :      536,994 руб.
СУММАРНЫЙ ПРОЦЕНТ ОШИБКИ  :       16.34%
```

Рисунок 4.9 – MAE на test, средняя цена, суммарный процент ошибки

```python
# Таблица ошибок по каждому тестовому примеру (это пункт 3 задания)
errors_df = pd.DataFrame({
    'Реальная цена': y_test.astype(int),
    'Предсказание':  pred_test.round(0).astype(int),
    '|Ошибка|':      errors_abs.round(0).astype(int),
    '% ошибки':      errors_pct.round(2),
})
print('Первые 20 примеров теста:')
errors_df.head(20)
```

Первые 20 примеров теста:

	Реальная цена	Предсказание	|Ошибка|	% ошибки
0	450000	430554	19446	4.320000
1	200000	137996	62004	31.000000
2	1085000	1044796	40204	3.710000
3	310000	339763	29763	9.600000
4	649000	621019	27981	4.310000
5	305000	251776	53224	17.450001
6	425000	423560	1440	0.340000
7	519000	413796	105204	20.270000
8	55000	71642	16642	30.260000
9	357000	358773	1773	0.500000
10	265000	208260	56740	21.410000
11	500000	421755	78245	15.650000
12	510000	452169	57831	11.340000
13	1035000	1077058	42058	4.060000
14	4400000	4213707	186293	4.230000
15	235000	167878	67122	28.559999
16	300000	196127	103873	34.619999
17	480000	412557	67443	14.050000
18	1247000	1268631	21631	1.730000
19	140000	201288	61288	43.779999

Рисунок 4.10 – Таблица ошибок по первым 20 примерам тестовой выборки

```
plt.figure(figsize=(10, 4))
plt.hist(np.clip(errors_pct, 0, 100), bins=50, color='steelblue', edgecolor='white')
plt.axvline(err_pct_test, color='red', linestyle='--',
            label=f'Суммарный % ошибки = {err_pct_test:.2f}%')
plt.xlabel('% ошибки на примере'); plt.ylabel('Количество')
plt.title('Распределение % ошибки на тесте'); plt.legend(); plt.grid(); plt.show()
```

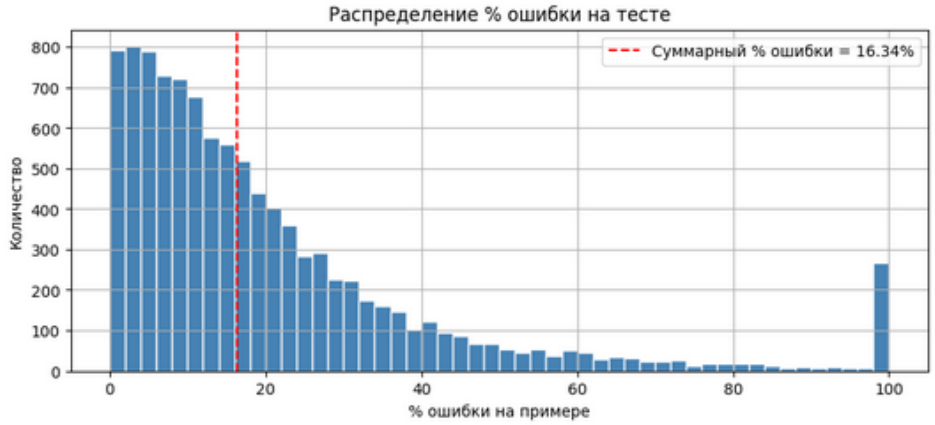


Рисунок 4.11 – Гистограмма распределения % ошибки на тесте

Итог: суммарный процент ошибки на тестовой выборке – 16.34% (MAE ≈ 87 730 руб. при средней цене ≈ 537 000 руб.). Ключевыми шагами стали жёсткая фильтрация мусорных объявлений, схлопывание редких категорий и нормировка цены на Y_MAX train-выборки.

Выводы: в ходе лабораторной работы были построены и обучены шесть полносвязных нейронных сетей для задачи регрессии (предсказание заработной платы по данным HeadHunter) на двух фреймворках – TensorFlow/Keras и PyTorch.